Question 1: SuperPoint 1

In this question we will look at the method SuperPoint [1], for establishing correspondences between a pair of images. Similar to UCN [2] which was discussed in the lectures, SuperPoint is a deep network to learn descriptors. However, SuperPoint simultaneously learns an interest point detector. In this question, you will generalize your understanding from the lectures to learn about SuperPoint, answer a few conceptual questions, test SuperPoint with pretrained models and observe the results.

In this question we will be using implementation of SuperPoint from a public repo (https://github.com/eric-yyjau/pytorch-superpoint).

Note: All paths in this question are relative to *./pytorch-superpoint/*. All scripts/commands should be run from that path in the terminal (by first `cd pytorch-superpoint/`).

Q1.1: Overall pipeline

The overall pipeline of SuperPoint is given in Fig. 3 of the paper. Please read Section 3 with a focus on Section 3.1 and 3.2 and answer:

(a) Why does SuperPoint use a shared encoder for interest point detection (Interest Point Decoder) and description (Descriptor Decoder)? (3 points)

SuperPoint uses a shared encoder to 'reduce the demonsionality of the image'. It makes it less comput intensive for the two decoders to the model is faster and easier to train.

(b) What is the motivation for the output shape of Interest Point Decoder being $H/8 \times W/8 \times 64$, instead of directly outputting an $H \times W$ heatmap from the last layer of the CNN-based decoder? (3 points)

The interest point decoder actually outputs H X W because the softmax and reshape functions are part of the decoder. The interest point detection head was designed with the 'explicit decoder' as mentioned in the paper to redue the computation of the model.

Q1.2: MagicPoint

The training pipeline is described in Section 4 and 5. In Section 4, the detector part is *trained on synthetic images dataset* with pre-defined interest points, in a fully supervised fashion. The resulting interest point detector is named **MagicPoint**. You will run inference for **MagicPoint** with model weights pretrained on synthetic images.

First, make sure the pretrained model in *configs/magicpoint_repeatability_heatmap.yaml* is:

`pretrained: 'logs/magicpoint_synth_t2/checkpoints/superPointNet_100000_checkpoint.pth.tar`
Run the following code in your terminal to run the inference over 100 images from HPatches dataset and export the results, following the experiment setup in Section 7:

```
python export.py export_descriptor configs/magicpoint_repeatability_heatmap.yaml magicpoint_syn_TEST_hpatches
```
Results are saved to: *logs/magicpoint_syn_TEST_hpatches/predictions/{%d}.npz*

Then, run the evaluation script to load the exported results and compute various metrics:

```
python evaluation.py logs/magicpoint_syn_TEST_hpatches/predictions --repeatibility --outputImg --homography --plotMatching
```
The following metrics will be printed out at the end of the run, similar to what is measured in Table 4 and explained in detail in Appendix A:

- Detector metrics:
  - Rep. (repeatability)
  - MLE (localization error)
- Descriptor metrics:
  - NN mAP (mean AP)
  - M. score (matching score)
- Homography accuracy under varying thresholds of pixel offsets (correctness_ave)

Please read the paper for detailed description of the metrics. Visualizations of interest points detected for a pair of images are saved to *logs/magicpoint_syn_TEST_hpatches/predictions/repeatibility/{%d}m.png*.

Please answer:

(a) Report the 4 metrics given above from the run. (2 points)

The metrics are as follows:

- Rep. - 0.437676
- MLE - 1.291547
- NN mAP - 0.699515
- M. score - 0.329822

(b) Please explain: what is repeatibility? (3 points)

Repeatability is the measure of the ability of the model to repeat or find interest points despite transformations and other alterations.

(c) Report detection visualizations for samples 2 and 5 by **MagicPoint** in the Markdown block below. (2 points)

`change the paths and run the block to display the images`

Then, you will use **SIFT** and **Harris corner detector** to run interest point detection on the same set of images:

```
python export_classical.py export_descriptor
configs/classical_descriptors.yaml sift_TEST_hpatches --correspondence
```

```
python evaluation.py logs/sift_TEST_hpatches/predictions --sift --
repeatibility --outputImg --homography --plotMatching
```

Visualizations of interest points detected for a pair of images are saved to *logs/sift_TEST_hpatches/predictions/repeatibility3/{%d}_{HARRIS, SIFT}.png*.

Please answer:

(d) Report the detection metrics (**repeatability** and **localization error**) from Harris detector from the above run. (2 points)

Rep - 0.402367
mLE - 1.18824654

(e) Report detection visualizations from samples 2 and 5 by **Harris detector** in the Markdown block below. (2 points)

`change the paths and run the block to display the images`

(f) Does **MagicPoint** perform better than **Harris detector**? Explain why or why not. (5 points)

The magic point repeatability is higher but also the localization error is higher. Magic point works better across while Harric Detector works better on the given. But overall they have fairly similar scores.

Q1.3: Homographic Adaptation and SuperPoint

MagicPoint is trained with full supervision on synthetic dataset, where interest points are defined as mostly corners of simple geometric primitatives (Fig. 4). However as a base detector, MagicPoint will not generalize well to natural images (e.g. HPatches) because of domain gap. As a result, Homographic Adaptation is proposed (see Section 5) to create pseudo-ground truth interest points on real images, and use them to finetune MagicPoint on real images in a self-supervised paradigm. The finetuned detector is named **SuperPoint**.

In this question, you will run inference of SuperPoint on HPatches, which is trained on COCO dataset.

You will use the same script to run inference, except that using SuperPoint finetuned on COCO instead of MagicPoint trained on synthetic images dataset.

First, change the pretrained model in *configs/magicpoint_repeatability_heatmap.yaml* via:

```
pretrained:
'logs/magicpoint_synth_t2/checkpoints/superPointNet_100000_checkpoint.pth.tar
==change to==>
```

```
pretrained:
'logs/superpoint_coco_heat2_0/checkpoints/superPointNet_170000_checkpoint.pth
```

Then run inference via:

```
python export.py export_descriptor
configs/magicpoint_repeatability_heatmap.yaml
superpoint_coco_TEST_hpatches
```

```
python evaluation.py logs/superpoint_coco_TEST_hpatches/predictions --
repeatibility --outputImg --homography --plotMatching
```

results are saved to: *logs/superpoint_coco_TEST_hpatches/predictions*

Please answer:

(a) Report the detection metrics (**repeatability** and **localization error**) from **SuperPoint** from the above run. (2 points)

REP - 0.63355 MLE - 1.0163788

(b) Does **SuperPoint** perform better than **MagicPoint** and **Harris detector**? Explain why or why not. (5 points)

SuperPoint does perform better - with higher repeatability scores and lower localization error scores

The descriptor is measured via descriptor metrics (**NN mAP** and **M. score**, see Section 7.3 and Appendix A). Additionally, detector and descriptor can be jointly evaluated via homography estimation, by computing homography from estimated correspondences, and measuring the average error of pixel offsets using the homography (see Appendix A).

(c) Report and compare SuperPoint and SIFT on **NN mAP**, **M. score** and **Homography accuracy under varying thresholds**. (3 points)

Super Point:

- mAP = 0.8083434
- mscore = 0.4626519
- correctness_ave [0.48 0.76 0.84 0.91 0.93 0.97]

SIFT:

- mAP = 0.7610357
- mscore = 0.2888203
- Average correctness: [0.69 0.79 0.87 0.87 0.89 0.93]

as we can see from above the scores for mAP and mscore super point are higher than those of SIFT

The scores for the homography estimation correctnes are comparable among the two with a higher limit for SIFT than Super Point

(d) Does SuperPoint perform better than SIFT on **Homography accuracy** for lower thresholds? Explain why or why not. (5 points)

SIFT performs worse for high thresholds but does better for lower thresholds. This may occur because Super Point's learned key points need more information to become more accurate, while sift uses by-pixel gradients so it can have more accuracy at lower thresholds.

Q1.4 Try SuperPoint on your own image pairs

In this question you are encouraged to try your own image pairs which have reasonable overlap of contents between two images. A sample image pair is given in *configs/magicpoint_repeatability_heatmap_your_images.yaml* → `data['im_path1']`, `data['im_path2']`

Please first run the following code to estimate and visualize correspondences of the sample pair from KITTI dataset. Visualization will be saved at *logs/superpoint_coco_TEST_yours/predictions/matching/0m.png*. If everything goes well, the image should look like:

Next, please change the image pair paths to a pair taken by yourself (under the assumption that reasonable amount of correspondences can be visually identified; also format has to be .jpg or .png). Also adjust the resize flag `data['preprocessing']['resize']` to be values propotional to your image aspect ratio and a size that fits into the gpu. You are also encourage to tweak the viaualization parameters of ratio/max number of all matches (variables `ratio_max` and `matches_max`) to make the visualization look good with reasonable amount of matches that is not too clustered.

```
python export.py export_descriptor
configs/magicpoint_repeatability_heatmap_your_images.yaml
superpoint_coco_TEST_yours

python evaluation.py logs/superpoint_coco_TEST_yours/predictions --
outputImg --plotMatchingOnly
```

(a) Report your visualizations below. (3 points)

`change the paths and run the block to display the image`

(b) **Bonus:** You are encouraged to take a second pair where it is more 'difficult' to establish abundant correspondences than the first set above. Explain how you created a scenario where correspondence is more challenging. Report the results again and explain your observation on the differences between results for the two pairs of images. (5 points)

`change the paths and run the block to display the other image`

`write your answer here`

Q2: Metric Learning for Fashion Images Retrival

In this problem you will train a CNN for learning embeddings for fashion images (e.g. clothing, shoes) with techniques from metric learning (e.g. Triplet loss, hard negative mining, distance functions). Once the network is trained, you will observe the application on similar image retrival, and evaluate design choices of the model.

Q2.1: Train a simple CNN and observe training and evaluations.

Run the following code within the notebook, which includes a full pipeline of learning embeddings for fashion images. By default the model will train for 2 epochs (i.e. twice over all training samples), with cosine similarity as distance function for embeddings, triplet margin loss as discussed in lectures, and semi-hard negative mining strategy. After each epoch, the pipeline will evaluate on a held-out test set to measure the accuracy with `accuracy_calculator.get_accuracy`.

Run the following pipeline, and answer the questions below.

In [4]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

import torchvision
```

```
from torchvision import datasets, transforms

from pytorch_metric_learning import distances, losses, miners, reducers, testers
from pytorch_metric_learning.utils.accuracy_calculator import AccuracyCalculator
from pytorch_metric_learning.distances import CosineSimilarity
from pytorch_metric_learning.utils import common_functions as c_f
from pytorch_metric_learning.utils.inference import InferenceModel, MatchFinder

import matplotlib.pyplot as plt
import numpy as np



### Define a network architecture ###
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.fc1 = nn.Linear(9216, 128)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
        x = torch.flatten(x, 1)
        # print(x.shape) # should be [batch_size, 9216]
        x = self.fc1(x)
        return x


### Do the training ###
def train(model, loss_func, mining_func, device, train_loader, optimizer, epoch):
    model.train()
    loss_list = []
    for batch_idx, (data, labels) in enumerate(train_loader):
        data, labels = data.to(device), labels.to(device)
        optimizer.zero_grad()
        embeddings = model(data)
        indices_tuple = mining_func(embeddings, labels)
        loss = loss_func(embeddings, labels, indices_tuple)
        loss.backward()
        optimizer.step()
        if batch_idx % 20 == 0:
            print(
                "Epoch {} Iteration {}: Loss = {}, Number of mined triplets = {}".f
                    epoch, batch_idx, loss, mining_func.num_triplets
                )
            )
        loss_list.append(loss.item())
    return loss_list
```

```
### convenient function from pytorch-metric-learning ###
def get_all_embeddings(dataset, model):
    tester = testers.BaseTester()
    return tester.get_all_embeddings(dataset, model)


### compute accuracy using AccuracyCalculator from pytorch-metric-learning ###
def test(train_set, test_set, model, accuracy_calculator):
    train_embeddings, train_labels = get_all_embeddings(train_set, model)
    test_embeddings, test_labels = get_all_embeddings(test_set, model)
    train_labels = train_labels.squeeze(1)
    test_labels = test_labels.squeeze(1)
    print("Computing accuracy")
    accuracies = accuracy_calculator.get_accuracy(
        test_embeddings, test_labels, train_embeddings, train_labels, False
    )
    print("Test set accuracy (Precision@1) = {:f}".format(accuracies["precision_at_
    return accuracies


device = torch.device("cuda")

transform = transforms.Compose(
    [transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))]
)

batch_size = 256

dataset1 = datasets.FashionMNIST(".", train=True, download=True, transform=transfor
dataset2 = datasets.FashionMNIST(".", train=False, transform=transform)
train_loader = torch.utils.data.DataLoader(dataset1, batch_size=256, shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset2, batch_size=256)

model = Net().to(device)

optimizer = optim.Adam(model.parameters(), lr=0.01)
num_epochs = 20


### pytorch-metric-learning stuff ###
distance = distances.CosineSimilarity()
#distance = distances.LpDistance()
reducer = reducers.ThresholdReducer(low=0)

margin_num = 0.2
loss_func = losses.TripletMarginLoss(margin=margin_num, distance=distance, reducer=
#loss_func = losses.ContrastiveLoss()
#loss_func = losses.ArcFaceLoss(num_classes = 10, embedding_size = 128)
mining_func = miners.TripletMarginMiner(
    margin=margin_num, distance=distance, type_of_triplets="semihard"
)


accuracy_calculator = AccuracyCalculator(include=("precision_at_1",), k=1)
### pytorch-metric-learning stuff ###
```

```
loss_list_all = []
accuracies_list = []
accuracies = test(dataset1, dataset2, model, accuracy_calculator)
accuracies_list.append((0, accuracies))

for epoch in range(1, num_epochs + 1):
    loss_list = train(model, loss_func, mining_func, device, train_loader, optimize
    accuracies = test(dataset1, dataset2, model, accuracy_calculator)
    loss_list_all += loss_list
    accuracies_list.append((len(loss_list_all), accuracies))
```

```
100%|██████████| 1875/1875 [00:04<00:00, 376.97it/s]
100%|██████████| 313/313 [00:01<00:00, 298.89it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.848700
Epoch 1 Iteration 0: Loss = 0.09601166844367981, Number of mined triplets = 718643
Epoch 1 Iteration 20: Loss = 0.1027473509311676, Number of mined triplets = 135797
Epoch 1 Iteration 40: Loss = 0.09707474708557129, Number of mined triplets = 108609
Epoch 1 Iteration 60: Loss = 0.09706522524356842, Number of mined triplets = 76480
Epoch 1 Iteration 80: Loss = 0.09555091708898544, Number of mined triplets = 79933
Epoch 1 Iteration 100: Loss = 0.09776715189218521, Number of mined triplets = 90312
Epoch 1 Iteration 120: Loss = 0.0935789942741394, Number of mined triplets = 71564
Epoch 1 Iteration 140: Loss = 0.09332029521465302, Number of mined triplets = 58536
Epoch 1 Iteration 160: Loss = 0.0953764095902443, Number of mined triplets = 68488
Epoch 1 Iteration 180: Loss = 0.09337008744478226, Number of mined triplets = 56145
Epoch 1 Iteration 200: Loss = 0.09336589276790619, Number of mined triplets = 62846
Epoch 1 Iteration 220: Loss = 0.09514831751585007, Number of mined triplets = 79437
100%|██████████| 1875/1875 [00:04<00:00, 375.92it/s]
100%|██████████| 313/313 [00:01<00:00, 308.33it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.858700
Epoch 2 Iteration 0: Loss = 0.09456679224967957, Number of mined triplets = 81663
Epoch 2 Iteration 20: Loss = 0.09626706689596176, Number of mined triplets = 60546
Epoch 2 Iteration 40: Loss = 0.09437581896781921, Number of mined triplets = 61096
Epoch 2 Iteration 60: Loss = 0.09145009517669678, Number of mined triplets = 56779
Epoch 2 Iteration 80: Loss = 0.09234306216239929, Number of mined triplets = 51314
Epoch 2 Iteration 100: Loss = 0.09182385355234146, Number of mined triplets = 49520
Epoch 2 Iteration 120: Loss = 0.09214957803487778, Number of mined triplets = 48802
Epoch 2 Iteration 140: Loss = 0.0940638929605484, Number of mined triplets = 59317
Epoch 2 Iteration 160: Loss = 0.09036440402269363, Number of mined triplets = 38969
Epoch 2 Iteration 180: Loss = 0.09283537417650223, Number of mined triplets = 47413
Epoch 2 Iteration 200: Loss = 0.09225418418645859, Number of mined triplets = 48286
Epoch 2 Iteration 220: Loss = 0.09255474805831909, Number of mined triplets = 60144
100%|██████████| 1875/1875 [00:04<00:00, 380.69it/s]
100%|██████████| 313/313 [00:01<00:00, 298.27it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.878200
Epoch 3 Iteration 0: Loss = 0.09538000077009201, Number of mined triplets = 49121
Epoch 3 Iteration 20: Loss = 0.09218332171440125, Number of mined triplets = 41269
Epoch 3 Iteration 40: Loss = 0.09100748598575592, Number of mined triplets = 47255
Epoch 3 Iteration 60: Loss = 0.09187362343072891, Number of mined triplets = 50949
Epoch 3 Iteration 80: Loss = 0.09327962249517441, Number of mined triplets = 50691
Epoch 3 Iteration 100: Loss = 0.09164820611476898, Number of mined triplets = 56246
Epoch 3 Iteration 120: Loss = 0.0918189287185669, Number of mined triplets = 41395
Epoch 3 Iteration 140: Loss = 0.09224377572536469, Number of mined triplets = 40719
Epoch 3 Iteration 160: Loss = 0.09127163887023926, Number of mined triplets = 38567
Epoch 3 Iteration 180: Loss = 0.09111621230840683, Number of mined triplets = 43924
Epoch 3 Iteration 200: Loss = 0.09167883545160294, Number of mined triplets = 43424
Epoch 3 Iteration 220: Loss = 0.09132750332355499, Number of mined triplets = 57044
100%|██████████| 1875/1875 [00:05<00:00, 372.95it/s]
100%|██████████| 313/313 [00:01<00:00, 292.17it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.882300
Epoch 4 Iteration 0: Loss = 0.09161005169153214, Number of mined triplets = 35255
Epoch 4 Iteration 20: Loss = 0.0880102589726448, Number of mined triplets = 32122
Epoch 4 Iteration 40: Loss = 0.09292244166135788, Number of mined triplets = 61174
Epoch 4 Iteration 60: Loss = 0.09167155623435974, Number of mined triplets = 35507
Epoch 4 Iteration 80: Loss = 0.08934507519006729, Number of mined triplets = 45113
Epoch 4 Iteration 100: Loss = 0.09035880863666534, Number of mined triplets = 43480
Epoch 4 Iteration 120: Loss = 0.09230291098356247, Number of mined triplets = 36065
Epoch 4 Iteration 140: Loss = 0.09203985333442688, Number of mined triplets = 45665
Epoch 4 Iteration 160: Loss = 0.0921000987291336, Number of mined triplets = 44722
Epoch 4 Iteration 180: Loss = 0.0928785502910614, Number of mined triplets = 46157
Epoch 4 Iteration 200: Loss = 0.08619492501020432, Number of mined triplets = 34434
Epoch 4 Iteration 220: Loss = 0.09054809808731079, Number of mined triplets = 40818
100%|██████████| 1875/1875 [00:05<00:00, 365.40it/s]
100%|██████████| 313/313 [00:00<00:00, 343.71it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.892000
Epoch 5 Iteration 0: Loss = 0.08951085060834885, Number of mined triplets = 48574
Epoch 5 Iteration 20: Loss = 0.08942602574825287, Number of mined triplets = 29770
Epoch 5 Iteration 40: Loss = 0.09227262437343597, Number of mined triplets = 37981
Epoch 5 Iteration 60: Loss = 0.09071990102529526, Number of mined triplets = 50766
Epoch 5 Iteration 80: Loss = 0.09007609635591507, Number of mined triplets = 33660
Epoch 5 Iteration 100: Loss = 0.09288141876459122, Number of mined triplets = 49243
Epoch 5 Iteration 120: Loss = 0.09061971306800842, Number of mined triplets = 32319
Epoch 5 Iteration 140: Loss = 0.08812081813812256, Number of mined triplets = 22285
Epoch 5 Iteration 160: Loss = 0.08951519429683685, Number of mined triplets = 38875
Epoch 5 Iteration 180: Loss = 0.09180989861488342, Number of mined triplets = 35790
Epoch 5 Iteration 200: Loss = 0.09159480035305023, Number of mined triplets = 42354
Epoch 5 Iteration 220: Loss = 0.09188168495893478, Number of mined triplets = 47484
100%|██████████| 1875/1875 [00:05<00:00, 368.16it/s]
100%|██████████| 313/313 [00:00<00:00, 356.02it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.889100
Epoch 6 Iteration 0: Loss = 0.09204962104558945, Number of mined triplets = 37850
Epoch 6 Iteration 20: Loss = 0.09024445712566376, Number of mined triplets = 28480
Epoch 6 Iteration 40: Loss = 0.09102482348680496, Number of mined triplets = 51628
Epoch 6 Iteration 60: Loss = 0.09282668679952621, Number of mined triplets = 46157
Epoch 6 Iteration 80: Loss = 0.0896550863981247, Number of mined triplets = 35495
Epoch 6 Iteration 100: Loss = 0.09098214656114578, Number of mined triplets = 38587
Epoch 6 Iteration 120: Loss = 0.09348377585411072, Number of mined triplets = 43974
Epoch 6 Iteration 140: Loss = 0.09151634573936462, Number of mined triplets = 37880
Epoch 6 Iteration 160: Loss = 0.09181395173072815, Number of mined triplets = 47781
Epoch 6 Iteration 180: Loss = 0.09071158617734909, Number of mined triplets = 39891
Epoch 6 Iteration 200: Loss = 0.09111811965703964, Number of mined triplets = 52624
Epoch 6 Iteration 220: Loss = 0.09151104092597961, Number of mined triplets = 37753
100%|██████████| 1875/1875 [00:04<00:00, 376.45it/s]
100%|██████████| 313/313 [00:01<00:00, 271.31it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.894200
Epoch 7 Iteration 0: Loss = 0.09249849617481232, Number of mined triplets = 38123
Epoch 7 Iteration 20: Loss = 0.09100210666656494, Number of mined triplets = 30478
Epoch 7 Iteration 40: Loss = 0.09108155220746994, Number of mined triplets = 39888
Epoch 7 Iteration 60: Loss = 0.08989428728818893, Number of mined triplets = 40414
Epoch 7 Iteration 80: Loss = 0.09264799952507019, Number of mined triplets = 44228
Epoch 7 Iteration 100: Loss = 0.08969292789697647, Number of mined triplets = 34651
Epoch 7 Iteration 120: Loss = 0.09127481281757355, Number of mined triplets = 49920
Epoch 7 Iteration 140: Loss = 0.09153737872838974, Number of mined triplets = 45040
Epoch 7 Iteration 160: Loss = 0.08910954743623734, Number of mined triplets = 45731
Epoch 7 Iteration 180: Loss = 0.091704823076725, Number of mined triplets = 38954
Epoch 7 Iteration 200: Loss = 0.09048306941986084, Number of mined triplets = 32300
Epoch 7 Iteration 220: Loss = 0.0896439328789711, Number of mined triplets = 32796
100%|██████████| 1875/1875 [00:04<00:00, 384.89it/s]
100%|██████████| 313/313 [00:01<00:00, 278.83it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.894300
Epoch 8 Iteration 0: Loss = 0.09103595465421677, Number of mined triplets = 40203
Epoch 8 Iteration 20: Loss = 0.08884701877832413, Number of mined triplets = 31675
Epoch 8 Iteration 40: Loss = 0.09045323729515076, Number of mined triplets = 27481
Epoch 8 Iteration 60: Loss = 0.09059685468673706, Number of mined triplets = 37139
Epoch 8 Iteration 80: Loss = 0.09153082966804504, Number of mined triplets = 40496
Epoch 8 Iteration 100: Loss = 0.09247232228517532, Number of mined triplets = 41330
Epoch 8 Iteration 120: Loss = 0.09219823032617569, Number of mined triplets = 54172
Epoch 8 Iteration 140: Loss = 0.09113185107707977, Number of mined triplets = 24797
Epoch 8 Iteration 160: Loss = 0.0901690125465393, Number of mined triplets = 49446
Epoch 8 Iteration 180: Loss = 0.09074568748474121, Number of mined triplets = 30690
Epoch 8 Iteration 200: Loss = 0.0906924456357956, Number of mined triplets = 39167
Epoch 8 Iteration 220: Loss = 0.09143704921007156, Number of mined triplets = 44524
100%|██████████| 1875/1875 [00:04<00:00, 379.26it/s]
100%|██████████| 313/313 [00:01<00:00, 298.27it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.895100
Epoch 9 Iteration 0: Loss = 0.08965715765953064, Number of mined triplets = 42398
Epoch 9 Iteration 20: Loss = 0.09143059700727463, Number of mined triplets = 28309
Epoch 9 Iteration 40: Loss = 0.08931728452444077, Number of mined triplets = 29601
Epoch 9 Iteration 60: Loss = 0.09190308302640915, Number of mined triplets = 38676
Epoch 9 Iteration 80: Loss = 0.0894826352596283, Number of mined triplets = 34119
Epoch 9 Iteration 100: Loss = 0.08888924866914749, Number of mined triplets = 39116
Epoch 9 Iteration 120: Loss = 0.08928866684436798, Number of mined triplets = 30330
Epoch 9 Iteration 140: Loss = 0.08913533389568329, Number of mined triplets = 37530
Epoch 9 Iteration 160: Loss = 0.0892968699336052, Number of mined triplets = 36437
Epoch 9 Iteration 180: Loss = 0.09317106008529663, Number of mined triplets = 53702
Epoch 9 Iteration 200: Loss = 0.08853311836719513, Number of mined triplets = 32972
Epoch 9 Iteration 220: Loss = 0.08918731659650803, Number of mined triplets = 27563
100%|██████████| 1875/1875 [00:05<00:00, 366.08it/s]
100%|██████████| 313/313 [00:00<00:00, 338.58it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.897000
Epoch 10 Iteration 0: Loss = 0.09006068855524063, Number of mined triplets = 30980
Epoch 10 Iteration 20: Loss = 0.09150604903697968, Number of mined triplets = 34608
Epoch 10 Iteration 40: Loss = 0.08858295530080795, Number of mined triplets = 30250
Epoch 10 Iteration 60: Loss = 0.09016673266887665, Number of mined triplets = 29869
Epoch 10 Iteration 80: Loss = 0.08920575678348541, Number of mined triplets = 29773
Epoch 10 Iteration 100: Loss = 0.09141363203525543, Number of mined triplets = 40355
Epoch 10 Iteration 120: Loss = 0.09059783816337585, Number of mined triplets = 34605
Epoch 10 Iteration 140: Loss = 0.09010758996009827, Number of mined triplets = 34578
Epoch 10 Iteration 160: Loss = 0.09029024094343185, Number of mined triplets = 29025
Epoch 10 Iteration 180: Loss = 0.08923086524009705, Number of mined triplets = 33193
Epoch 10 Iteration 200: Loss = 0.0903983861207962, Number of mined triplets = 43933
Epoch 10 Iteration 220: Loss = 0.08939302712678909, Number of mined triplets = 40408
100%|██████████| 1875/1875 [00:05<00:00, 373.12it/s]
100%|██████████| 313/313 [00:01<00:00, 281.88it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.894900
Epoch 11 Iteration 0: Loss = 0.08961492031812668, Number of mined triplets = 33064
Epoch 11 Iteration 20: Loss = 0.08942000567913055, Number of mined triplets = 40468
Epoch 11 Iteration 40: Loss = 0.0898040384054184, Number of mined triplets = 31796
Epoch 11 Iteration 60: Loss = 0.09039969742298126, Number of mined triplets = 25560
Epoch 11 Iteration 80: Loss = 0.08997036516666412, Number of mined triplets = 38749
Epoch 11 Iteration 100: Loss = 0.0910043865442276, Number of mined triplets = 49329
Epoch 11 Iteration 120: Loss = 0.09069927781820297, Number of mined triplets = 32611
Epoch 11 Iteration 140: Loss = 0.0921146422624588, Number of mined triplets = 43638
Epoch 11 Iteration 160: Loss = 0.08759114891290665, Number of mined triplets = 28858
Epoch 11 Iteration 180: Loss = 0.09126666933298111, Number of mined triplets = 45832
Epoch 11 Iteration 200: Loss = 0.08881498873233795, Number of mined triplets = 31589
Epoch 11 Iteration 220: Loss = 0.08801373839378357, Number of mined triplets = 30595
100%|██████████| 1875/1875 [00:05<00:00, 357.90it/s]
100%|██████████| 313/313 [00:00<00:00, 317.12it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.894500
Epoch 12 Iteration 0: Loss = 0.08862418681383133, Number of mined triplets = 32954
Epoch 12 Iteration 20: Loss = 0.08751196414232254, Number of mined triplets = 25677
Epoch 12 Iteration 40: Loss = 0.09043210744857788, Number of mined triplets = 42550
Epoch 12 Iteration 60: Loss = 0.09104561060667038, Number of mined triplets = 44349
Epoch 12 Iteration 80: Loss = 0.0886300802230835, Number of mined triplets = 27210
Epoch 12 Iteration 100: Loss = 0.08870796114206314, Number of mined triplets = 31096
Epoch 12 Iteration 120: Loss = 0.08945831656455994, Number of mined triplets = 38710
Epoch 12 Iteration 140: Loss = 0.09143800288438797, Number of mined triplets = 33600
Epoch 12 Iteration 160: Loss = 0.08827222883701324, Number of mined triplets = 31559
Epoch 12 Iteration 180: Loss = 0.0879533439874649, Number of mined triplets = 27684
Epoch 12 Iteration 200: Loss = 0.09149283915758133, Number of mined triplets = 44878
Epoch 12 Iteration 220: Loss = 0.08954046666622162, Number of mined triplets = 32892
100%|██████████| 1875/1875 [00:05<00:00, 362.99it/s]
100%|██████████| 313/313 [00:01<00:00, 299.03it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.896400
Epoch 13 Iteration 0: Loss = 0.08949583023786545, Number of mined triplets = 29306
Epoch 13 Iteration 20: Loss = 0.08917640894651413, Number of mined triplets = 33037
Epoch 13 Iteration 40: Loss = 0.08793838322162628, Number of mined triplets = 22389
Epoch 13 Iteration 60: Loss = 0.08993836492300034, Number of mined triplets = 34896
Epoch 13 Iteration 80: Loss = 0.08942362666130066, Number of mined triplets = 29807
Epoch 13 Iteration 100: Loss = 0.08802402764558792, Number of mined triplets = 24388
Epoch 13 Iteration 120: Loss = 0.09156790375709534, Number of mined triplets = 41047
Epoch 13 Iteration 140: Loss = 0.08961518108844757, Number of mined triplets = 31287
Epoch 13 Iteration 160: Loss = 0.08903200924396515, Number of mined triplets = 36435
Epoch 13 Iteration 180: Loss = 0.0899481549859047, Number of mined triplets = 41825
Epoch 13 Iteration 200: Loss = 0.08728491514921188, Number of mined triplets = 29813
Epoch 13 Iteration 220: Loss = 0.09150221198797226, Number of mined triplets = 33884
100%|██████████| 1875/1875 [00:04<00:00, 378.64it/s]
100%|██████████| 313/313 [00:01<00:00, 294.77it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.899000
Epoch 14 Iteration 0: Loss = 0.08744338154792786, Number of mined triplets = 24873
Epoch 14 Iteration 20: Loss = 0.08786725252866745, Number of mined triplets = 31885
Epoch 14 Iteration 40: Loss = 0.08821550756692886, Number of mined triplets = 28000
Epoch 14 Iteration 60: Loss = 0.09016120433807373, Number of mined triplets = 39073
Epoch 14 Iteration 80: Loss = 0.08847212046384811, Number of mined triplets = 30620
Epoch 14 Iteration 100: Loss = 0.09012360870838165, Number of mined triplets = 24573
Epoch 14 Iteration 120: Loss = 0.08933665603399277, Number of mined triplets = 30468
Epoch 14 Iteration 140: Loss = 0.08960142731666565, Number of mined triplets = 36566
Epoch 14 Iteration 160: Loss = 0.08827736973762512, Number of mined triplets = 23613
Epoch 14 Iteration 180: Loss = 0.09076045453548431, Number of mined triplets = 38769
Epoch 14 Iteration 200: Loss = 0.08875549584627151, Number of mined triplets = 34493
Epoch 14 Iteration 220: Loss = 0.09083274006843567, Number of mined triplets = 32487
100%|██████████| 1875/1875 [00:04<00:00, 375.74it/s]
100%|██████████| 313/313 [00:01<00:00, 284.81it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.899000
Epoch 15 Iteration 0: Loss = 0.08901497721672058, Number of mined triplets = 23470
Epoch 15 Iteration 20: Loss = 0.091041199862957, Number of mined triplets = 34474
Epoch 15 Iteration 40: Loss = 0.09254496544599533, Number of mined triplets = 41116
Epoch 15 Iteration 60: Loss = 0.0895179808139801, Number of mined triplets = 35594
Epoch 15 Iteration 80: Loss = 0.09077699482440948, Number of mined triplets = 30065
Epoch 15 Iteration 100: Loss = 0.0892399474978447, Number of mined triplets = 34798
Epoch 15 Iteration 120: Loss = 0.08931080251932144, Number of mined triplets = 35814
Epoch 15 Iteration 140: Loss = 0.09152180701494217, Number of mined triplets = 42954
Epoch 15 Iteration 160: Loss = 0.08660667389631271, Number of mined triplets = 30069
Epoch 15 Iteration 180: Loss = 0.08814294636249542, Number of mined triplets = 29525
Epoch 15 Iteration 200: Loss = 0.09048350900411606, Number of mined triplets = 24989
Epoch 15 Iteration 220: Loss = 0.08815043419599533, Number of mined triplets = 35742
100%|██████████| 1875/1875 [00:05<00:00, 355.88it/s]
100%|██████████| 313/313 [00:01<00:00, 271.49it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.899600
Epoch 16 Iteration 0: Loss = 0.08942347019910812, Number of mined triplets = 37102
Epoch 16 Iteration 20: Loss = 0.08641035109758377, Number of mined triplets = 20520
Epoch 16 Iteration 40: Loss = 0.08995755761861801, Number of mined triplets = 35779
Epoch 16 Iteration 60: Loss = 0.08825654536485672, Number of mined triplets = 35004
Epoch 16 Iteration 80: Loss = 0.09055153280496597, Number of mined triplets = 34155
Epoch 16 Iteration 100: Loss = 0.08742665499448776, Number of mined triplets = 27691
Epoch 16 Iteration 120: Loss = 0.08927741646766663, Number of mined triplets = 40514
Epoch 16 Iteration 140: Loss = 0.08846305310726166, Number of mined triplets = 18372
Epoch 16 Iteration 160: Loss = 0.09111762046813965, Number of mined triplets = 31207
Epoch 16 Iteration 180: Loss = 0.09064223617315292, Number of mined triplets = 39802
Epoch 16 Iteration 200: Loss = 0.0871889740228653, Number of mined triplets = 20038
Epoch 16 Iteration 220: Loss = 0.08865363895893097, Number of mined triplets = 37061
100%|██████████| 1875/1875 [00:05<00:00, 363.22it/s]
100%|██████████| 313/313 [00:00<00:00, 337.24it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.899200
Epoch 17 Iteration 0: Loss = 0.08842536062002182, Number of mined triplets = 23768
Epoch 17 Iteration 20: Loss = 0.08913271874189377, Number of mined triplets = 39779
Epoch 17 Iteration 40: Loss = 0.08912871032953262, Number of mined triplets = 28759
Epoch 17 Iteration 60: Loss = 0.08866234868764877, Number of mined triplets = 34415
Epoch 17 Iteration 80: Loss = 0.0887724980711937, Number of mined triplets = 32028
Epoch 17 Iteration 100: Loss = 0.0903756245970726, Number of mined triplets = 32321
Epoch 17 Iteration 120: Loss = 0.08782333135604858, Number of mined triplets = 26848
Epoch 17 Iteration 140: Loss = 0.08752909302711487, Number of mined triplets = 23681
Epoch 17 Iteration 160: Loss = 0.09099499881267548, Number of mined triplets = 37271
Epoch 17 Iteration 180: Loss = 0.09317634254693985, Number of mined triplets = 45706
Epoch 17 Iteration 200: Loss = 0.08974921703338623, Number of mined triplets = 32358
Epoch 17 Iteration 220: Loss = 0.09039165079593658, Number of mined triplets = 44667
100%|██████████| 1875/1875 [00:05<00:00, 374.17it/s]
100%|██████████| 313/313 [00:01<00:00, 308.71it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.903100
Epoch 18 Iteration 0: Loss = 0.08715300261974335, Number of mined triplets = 25907
Epoch 18 Iteration 20: Loss = 0.09006457775831223, Number of mined triplets = 23594
Epoch 18 Iteration 40: Loss = 0.08728714287281036, Number of mined triplets = 30641
Epoch 18 Iteration 60: Loss = 0.08906209468841553, Number of mined triplets = 24175
Epoch 18 Iteration 80: Loss = 0.08671003580093384, Number of mined triplets = 26368
Epoch 18 Iteration 100: Loss = 0.08943066745996475, Number of mined triplets = 31411
Epoch 18 Iteration 120: Loss = 0.09062176942825317, Number of mined triplets = 23723
Epoch 18 Iteration 140: Loss = 0.08662614226341248, Number of mined triplets = 31304
Epoch 18 Iteration 160: Loss = 0.08958791196346283, Number of mined triplets = 36161
Epoch 18 Iteration 180: Loss = 0.09045131504535675, Number of mined triplets = 34229
Epoch 18 Iteration 200: Loss = 0.0893605649471283, Number of mined triplets = 27381
Epoch 18 Iteration 220: Loss = 0.08851852267980576, Number of mined triplets = 33347
100%|██████████| 1875/1875 [00:05<00:00, 368.38it/s]
100%|██████████| 313/313 [00:00<00:00, 342.31it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.900700
Epoch 19 Iteration 0: Loss = 0.09188174456357956, Number of mined triplets = 37517
Epoch 19 Iteration 20: Loss = 0.08734867721796036, Number of mined triplets = 24056
Epoch 19 Iteration 40: Loss = 0.08664269000291824, Number of mined triplets = 22989
Epoch 19 Iteration 60: Loss = 0.08999379724264145, Number of mined triplets = 20979
Epoch 19 Iteration 80: Loss = 0.08945921808481216, Number of mined triplets = 27681
Epoch 19 Iteration 100: Loss = 0.09050983935594559, Number of mined triplets = 28713
Epoch 19 Iteration 120: Loss = 0.08968795090913773, Number of mined triplets = 39149
Epoch 19 Iteration 140: Loss = 0.09094025939702988, Number of mined triplets = 36014
Epoch 19 Iteration 160: Loss = 0.09048137068748474, Number of mined triplets = 30143
Epoch 19 Iteration 180: Loss = 0.08916866034269333, Number of mined triplets = 32669
Epoch 19 Iteration 200: Loss = 0.08945170044898987, Number of mined triplets = 31417
Epoch 19 Iteration 220: Loss = 0.09073152393102646, Number of mined triplets = 40380
100%|██████████| 1875/1875 [00:04<00:00, 383.44it/s]
100%|██████████| 313/313 [00:01<00:00, 269.67it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.899300
Epoch 20 Iteration 0: Loss = 0.0894089862704277, Number of mined triplets = 28354
Epoch 20 Iteration 20: Loss = 0.0904294103384018, Number of mined triplets = 28447
Epoch 20 Iteration 40: Loss = 0.08710644394159317, Number of mined triplets = 27826
Epoch 20 Iteration 60: Loss = 0.08620312064886093, Number of mined triplets = 26467
Epoch 20 Iteration 80: Loss = 0.08894181251525879, Number of mined triplets = 22870
Epoch 20 Iteration 100: Loss = 0.08825760334730148, Number of mined triplets = 26104
Epoch 20 Iteration 120: Loss = 0.08616718649864197, Number of mined triplets = 20289
Epoch 20 Iteration 140: Loss = 0.08854282647371292, Number of mined triplets = 28003
Epoch 20 Iteration 160: Loss = 0.08735880255699158, Number of mined triplets = 24664
Epoch 20 Iteration 180: Loss = 0.09116256237030029, Number of mined triplets = 34310
Epoch 20 Iteration 200: Loss = 0.08832129091024399, Number of mined triplets = 28552
Epoch 20 Iteration 220: Loss = 0.08918045461177826, Number of mined triplets = 29911
100%|██████████| 1875/1875 [00:05<00:00, 370.94it/s]
100%|██████████| 313/313 [00:01<00:00, 298.11it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.899800
```

(a) Report the final test set accuracy. (2 points)

The final test set accuracy is %87.43 (two epochs), or %89.98 (20 epochs)

(b) Two variables are returned from the above code: `loss_list_all` which is a list of losses at each training iteration; `accuracies_list` which is a list of tuples for accuracies (precision) from testing after each epoch, with each tuple being `(ITERATION, {'precision_at_1': SOME NUMBER})`. You will plot a double-y-axis figure of training losses and testing accuracies from those two variables, with the x-axis being the training iterations, and the left y-axis being the training losses, right y-axis being the testing accuracies. A reference figure is given below; your plot does not have to be identical to it, but should ideally include two curves for losses and accuracies with the x axis being the iters, and necessary labels & legends. (5 points)

In [2]:
```python
import matplotlib.pyplot as plt

loss_iters = list(range(len(loss_list_all)))
loss_vals = loss_list_all

acc_iters = [x[0] for x in accuracies_list]
acc_vals = [x[1]['precision_at_1'] for x in accuracies_list]

fig, ax1 = plt.subplots()

# Left: training loss
ax1.set_xlabel('training iter')
ax1.set_ylabel('training loss', color='blue')
ax1.plot(loss_iters, loss_vals, color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# Right: testing accuracy
ax2 = ax1.twinx()
ax2.set_ylabel('testing accuracy', color='green')
ax2.plot(acc_iters, acc_vals, 'o-', color='green')
ax2.tick_params(axis='y', labelcolor='green')

plt.title('Training Loss and Testing Accuracy')
plt.show()
```

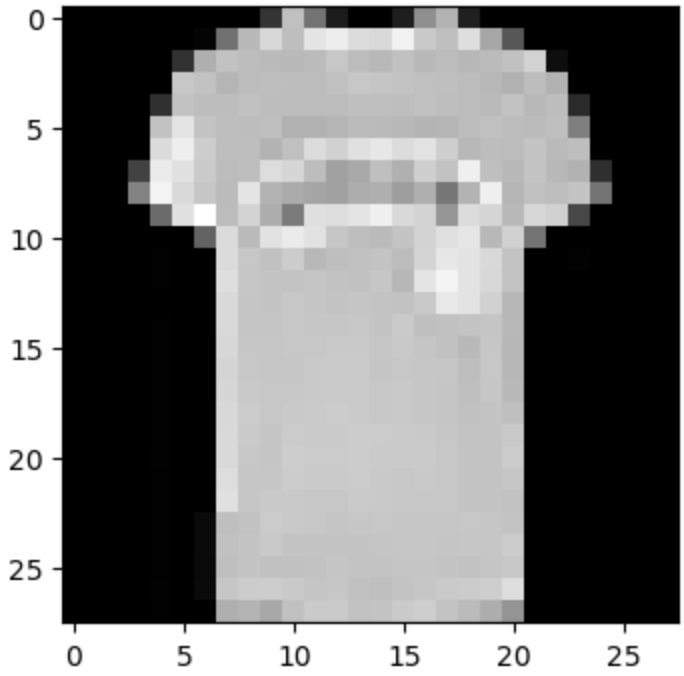
(c) You will then test the trained model on image retrival task, to retrieve the nearest images for a few test query images. Run the following code and submit the results. Make sure results for >=2 samples are visible in your converted PDF submission. (2 points)

In [3]:
```
def print_decision(is_match):
    if is_match:
        print("Same class")
    else:
        print("Different class")

mean = [0.1307]
std = [0.3081]
inv_normalize = transforms.Normalize(
    mean=[-m / s for m, s in zip(mean, std)], std=[1 / s for s in std]
)

def imshow(img, figsize=(8, 4)):
    img = inv_normalize(img)
    npimg = img.numpy()
    plt.figure(figsize=figsize)
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()


THRESHOLD = 0.7
match_finder = MatchFinder(distance=CosineSimilarity(), threshold=THRESHOLD)
inference_model = InferenceModel(model, match_finder=match_finder)
labels_to_indices = c_f.get_labels_to_indices(dataset2.targets)
```

```
# get 10 nearest neighbors for an input image
inference_model.train_knn(dataset2)
for label in range(0, 10):
    img_type = labels_to_indices[label]
    img = dataset2[img_type[0]][0].unsqueeze(0)
    print("query image")
    imshow(torchvision.utils.make_grid(img))
    distances, indices = inference_model.get_nearest_neighbors(img, k=10)
    nearest_imgs = [dataset2[i][0] for i in indices.cpu()[0]]
    print("nearest images")
    imshow(torchvision.utils.make_grid(nearest_imgs))
```

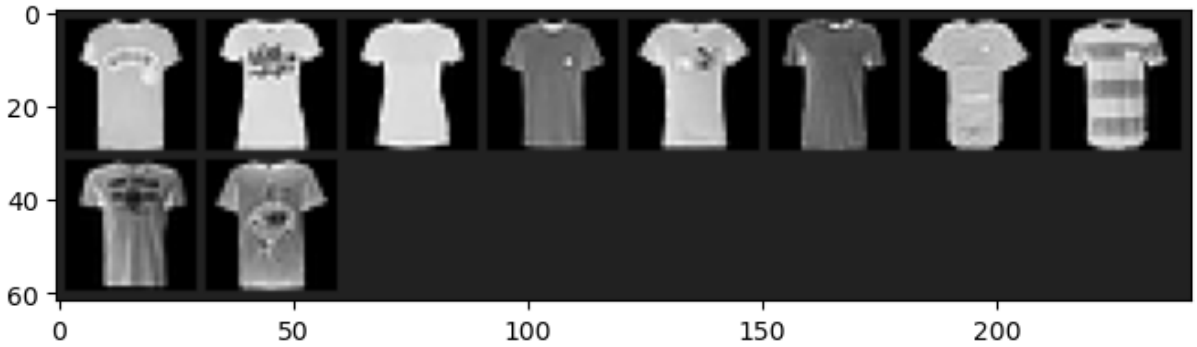
```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

```
query image
```



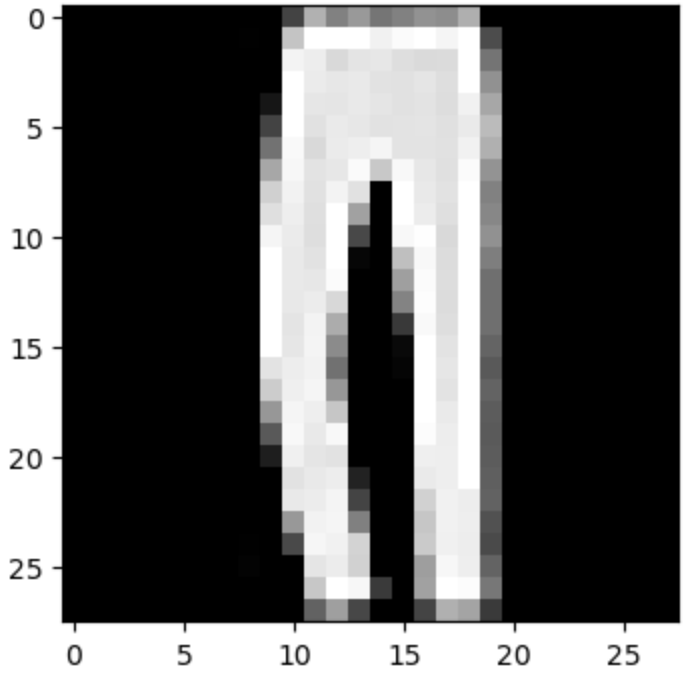

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

```
nearest images
```



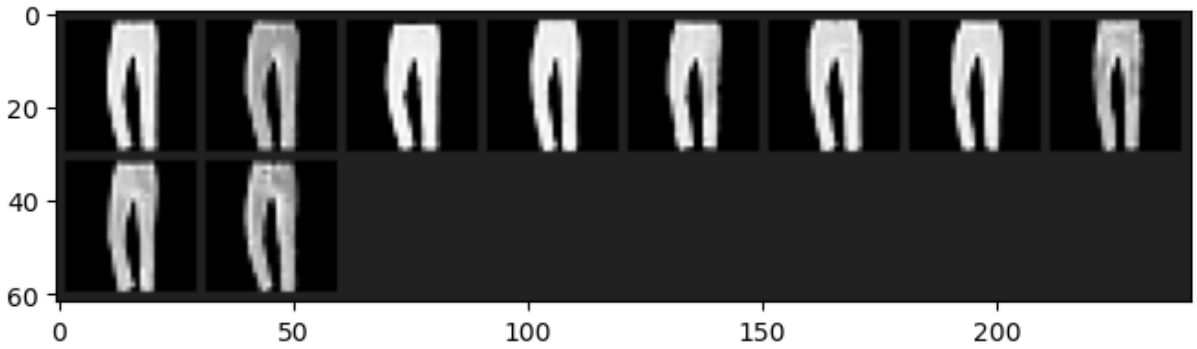

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

```
query image
```



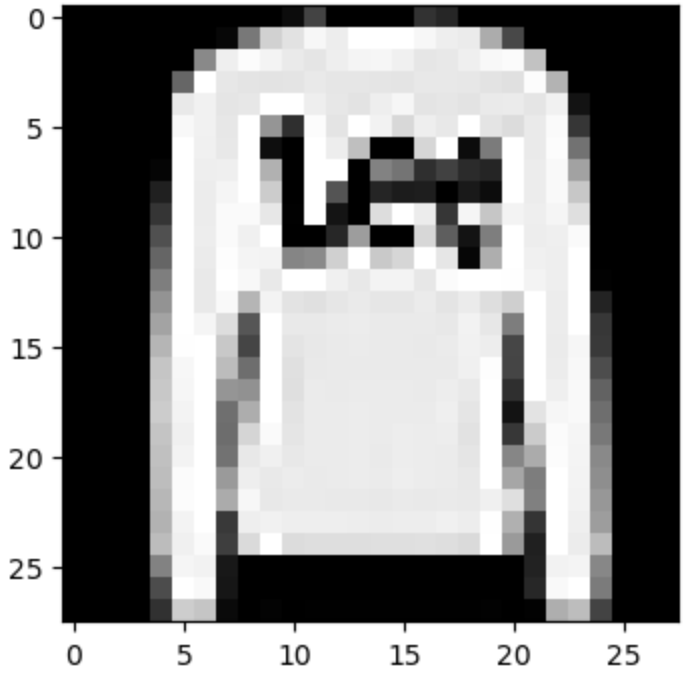

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

nearest images



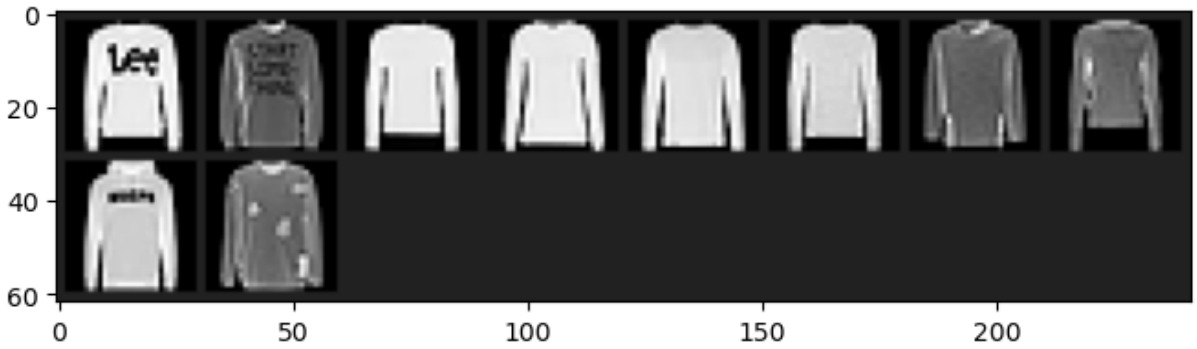

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

query image



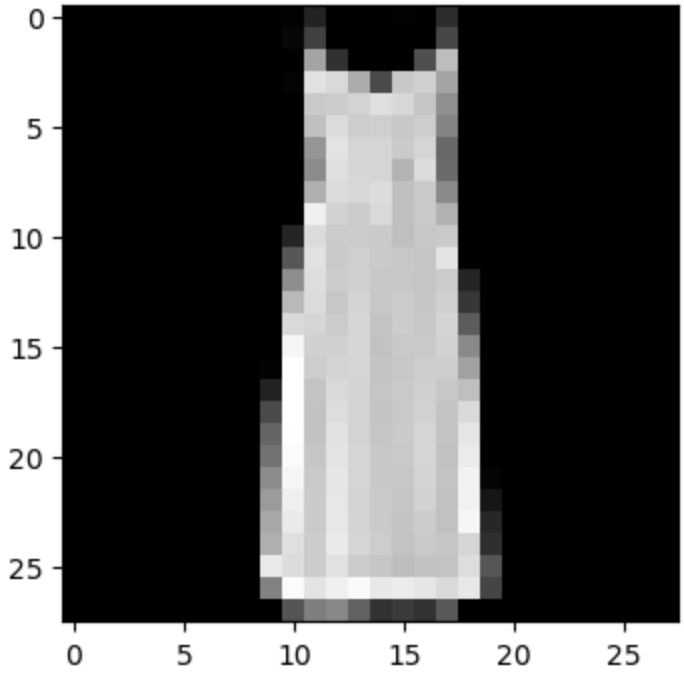
nearest images

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```



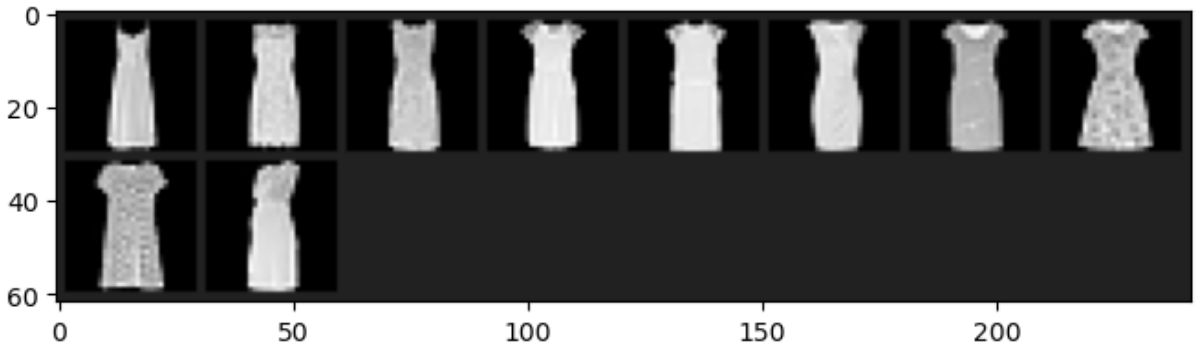

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

query image



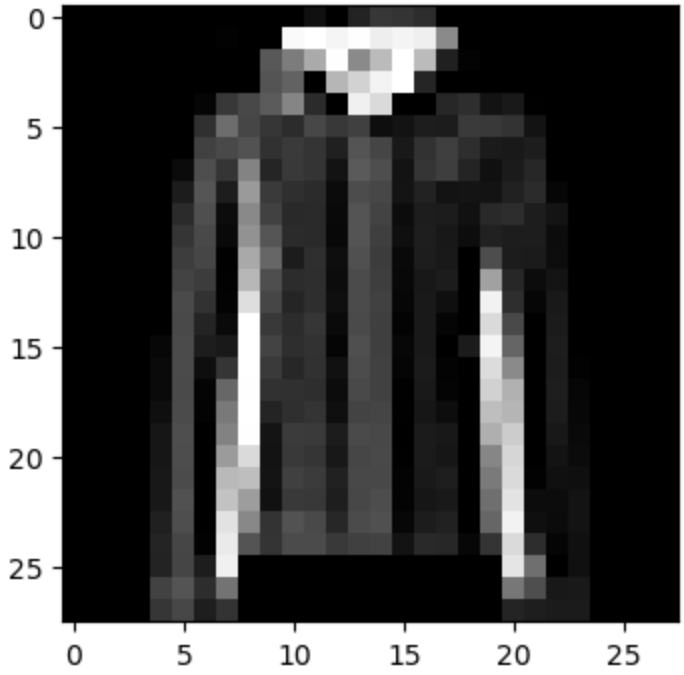

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

nearest images



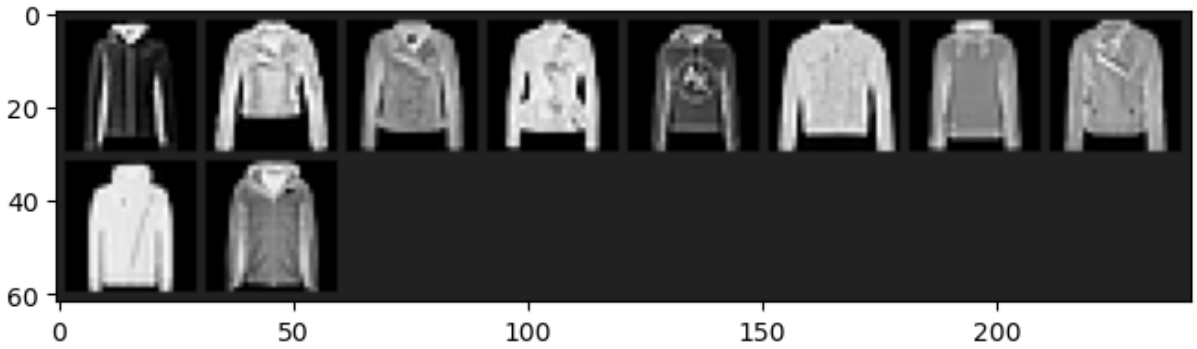

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

query image



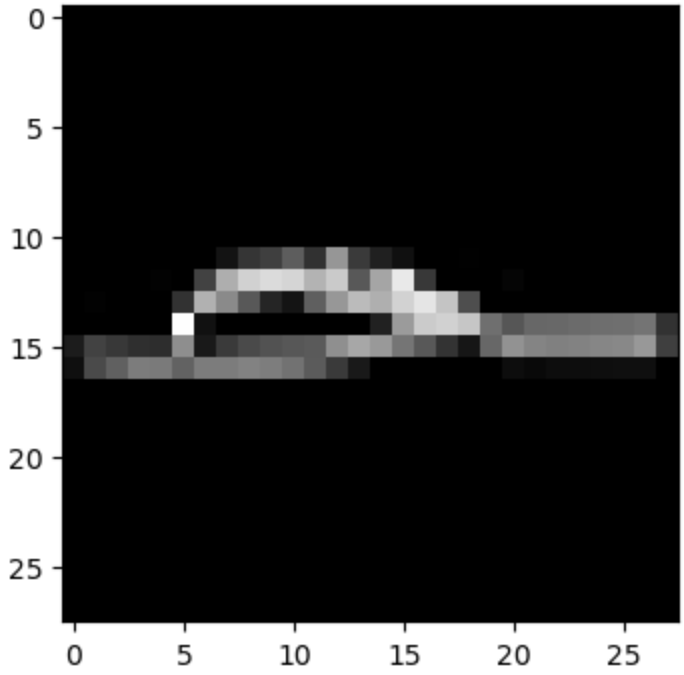
nearest images

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```



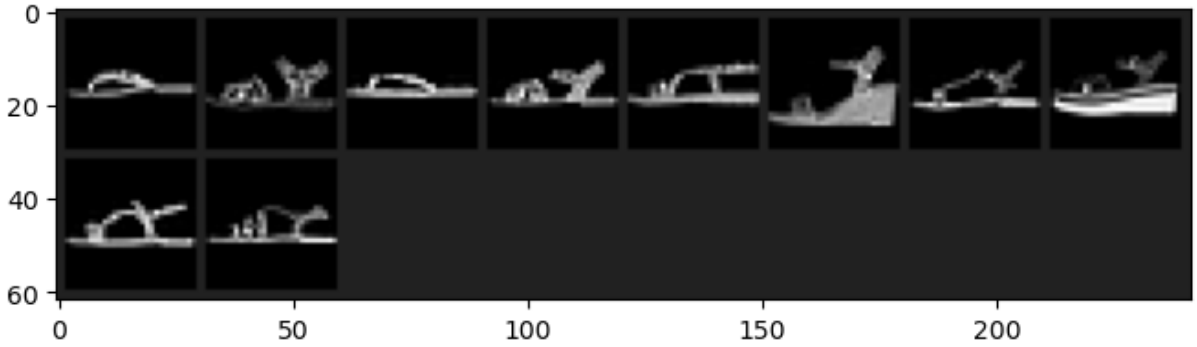

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

query image



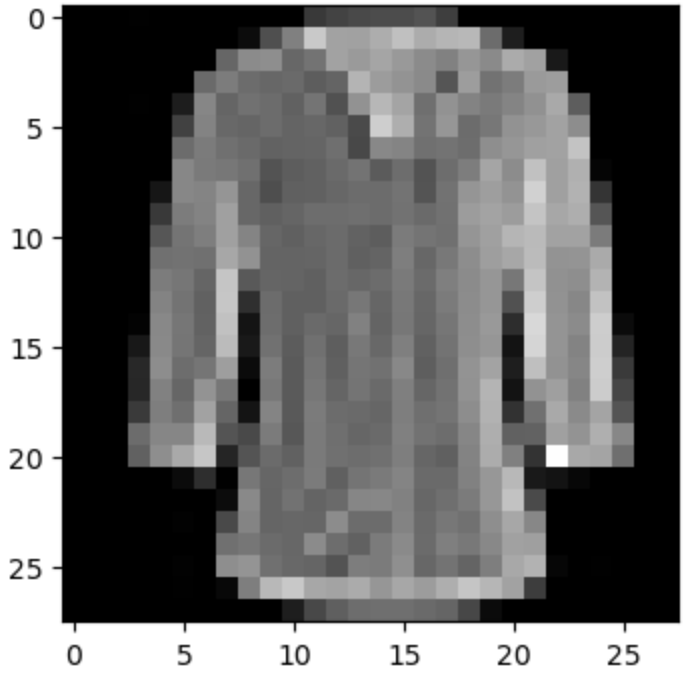

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

nearest images



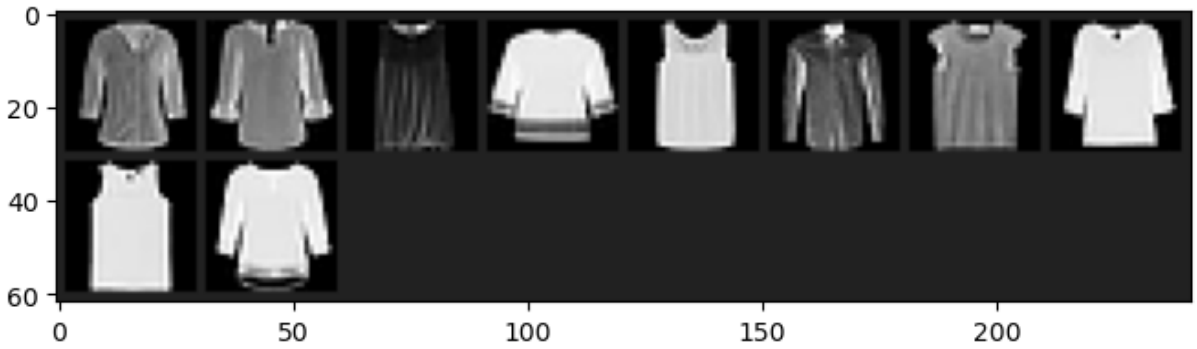

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

query image



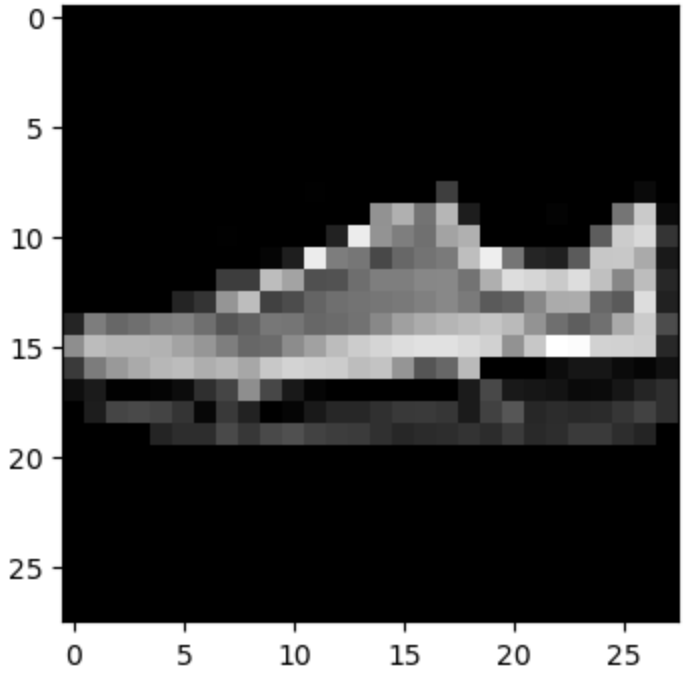
nearest images

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```



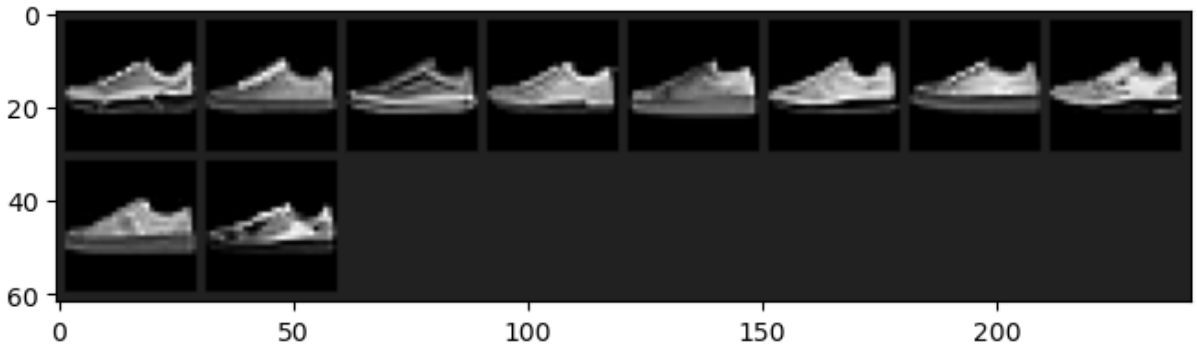

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

query image



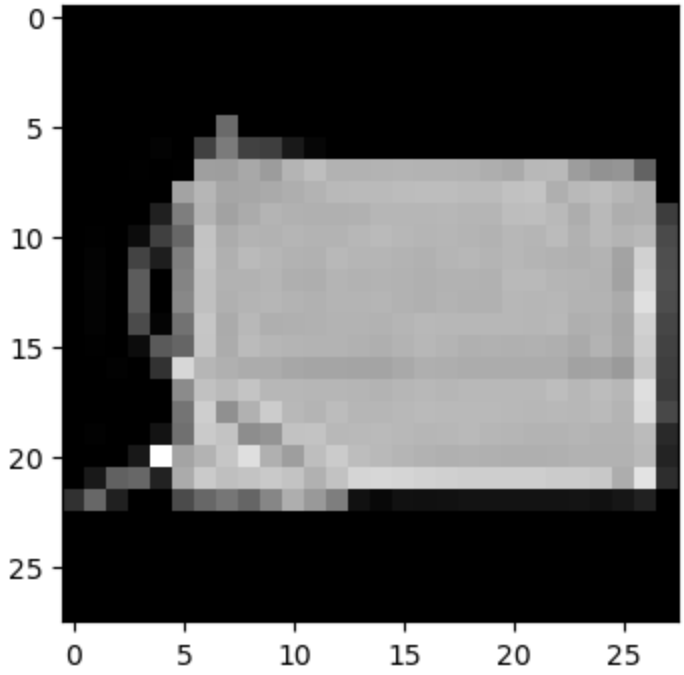

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

```
nearest images
```



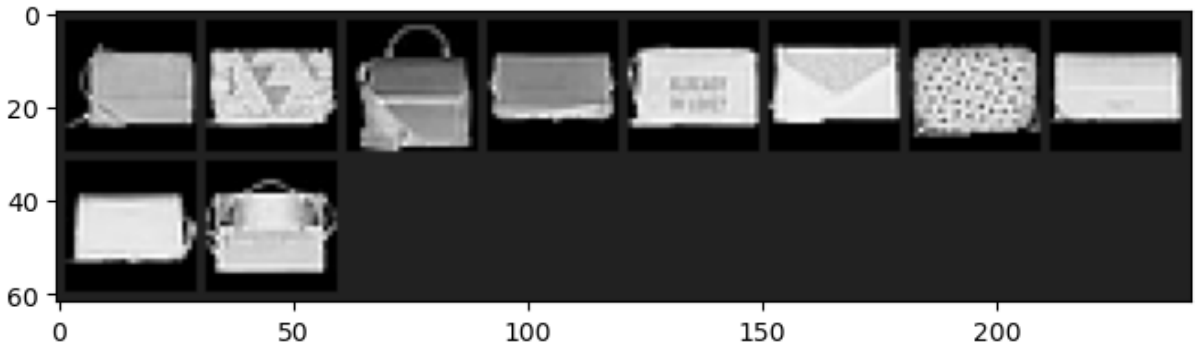

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

```
query image
```



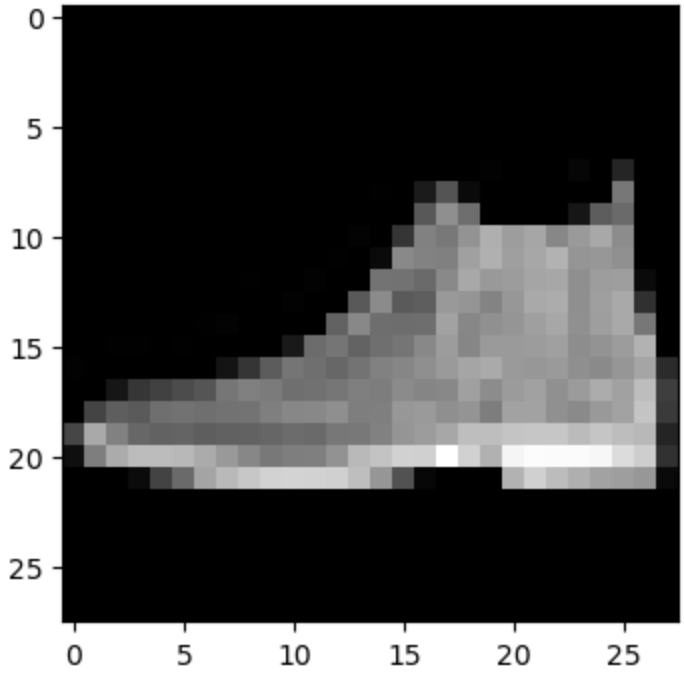

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

nearest images



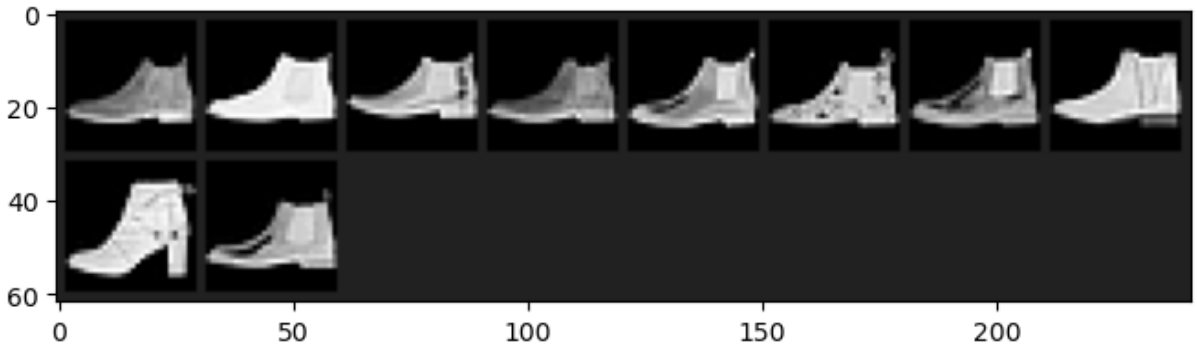

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```

query image




```
nearest images
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers).
```



(d) Tweak the pipeline, by

- Train for 20 epochs instead of 2;

Re-train the model. Plot the two curves again from (b) with the retrained model in the updated setting. (2 points)

In [5]:
```
import matplotlib.pyplot as plt

loss_iters = list(range(len(loss_list_all)))
loss_vals = loss_list_all

acc_iters = [x[0] for x in accuracies_list]
acc_vals = [x[1]['precision_at_1'] for x in accuracies_list]

fig, ax1 = plt.subplots()

# Left: training loss
ax1.set_xlabel('training iter')
```

```
ax1.set_ylabel('training loss', color='blue')
ax1.plot(loss_iters, loss_vals, color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# Right: testing accuracy
ax2 = ax1.twinx()
ax2.set_ylabel('testing accuracy', color='green')
ax2.plot(acc_iters, acc_vals, 'o-', color='green')
ax2.tick_params(axis='y', labelcolor='green')

plt.title('Training Loss and Testing Accuracy')
plt.show()
```

`include the image here, by downloading from the notebook and uploading it into the ./images folder, modify the path and run the block to reveal the image:`

(e) Tweak the model by:

- Changing `self.conv1` to output 8 channels instead of 32;
- Removing `self.conv2` and its activation function, and modify the input dimension of `self.fc1` from `9216` to some other number to make the layers fit together;
- Training for 20 epochs instead of 2;

1. Plot the curves (training losses and validation accuracies) again; (1 points)

2. Compare with the curves from (d): What difference can you observe between results (training losses and validation accuracies) of the two models? Hint: think about convergence rates, final value, etc. (2 points)

3. Explain how the change of model design lead to the observed differences. (5 points)

4. Copy the class of your new model in the code block below; (3 points)

1. `include the image here, by downloading from the notebook and uploading it into the ./images folder, modify the path and run the block to reveal the image:`

2. The results were much less stable, as makes sense when taking away a lot of parameters

3. As stated above, we drop out a very large portion of parameters so a lot less convolutional computation in the model

In [6]:
```
# your code here
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, 1)
        #self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.fc1 = nn.Linear(1352, 128)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        #x = self.conv2(x)
        #x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
        x = torch.flatten(x, 1)
        # print(x.shape) # should be [batch_size, 9216]
        x = self.fc1(x)
        return x


### Do the training ###
def train(model, loss_func, mining_func, device, train_loader, optimizer, epoch):
    model.train()
    loss_list = []
    for batch_idx, (data, labels) in enumerate(train_loader):
        data, labels = data.to(device), labels.to(device)
        optimizer.zero_grad()
        embeddings = model(data)
        indices_tuple = mining_func(embeddings, labels)
        loss = loss_func(embeddings, labels, indices_tuple)
```

```
        loss.backward()
        optimizer.step()
        if batch_idx % 20 == 0:
            print(
                "Epoch {} Iteration {}: Loss = {}, Number of mined triplets = {}".f
                    epoch, batch_idx, loss, mining_func.num_triplets
                )
            )
        loss_list.append(loss.item())
    return loss_list


### convenient function from pytorch-metric-learning ###
def get_all_embeddings(dataset, model):
    tester = testers.BaseTester()
    return tester.get_all_embeddings(dataset, model)


### compute accuracy using AccuracyCalculator from pytorch-metric-learning ###
def test(train_set, test_set, model, accuracy_calculator):
    train_embeddings, train_labels = get_all_embeddings(train_set, model)
    test_embeddings, test_labels = get_all_embeddings(test_set, model)
    train_labels = train_labels.squeeze(1)
    test_labels = test_labels.squeeze(1)
    print("Computing accuracy")
    accuracies = accuracy_calculator.get_accuracy(
        test_embeddings, test_labels, train_embeddings, train_labels, False
    )
    print("Test set accuracy (Precision@1) = {:f}".format(accuracies["precision_at_
    return accuracies


device = torch.device("cuda")

transform = transforms.Compose(
    [transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))]
)

batch_size = 256

dataset1 = datasets.FashionMNIST(".", train=True, download=True, transform=transfor
dataset2 = datasets.FashionMNIST(".", train=False, transform=transform)
train_loader = torch.utils.data.DataLoader(dataset1, batch_size=256, shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset2, batch_size=256)

model = Net().to(device)

optimizer = optim.Adam(model.parameters(), lr=0.01)
num_epochs = 20


### pytorch-metric-learning stuff ###
distance = distances.CosineSimilarity()
#distance = distances.LpDistance()
reducer = reducers.ThresholdReducer(low=0)
```

```
margin_num = 0.2
loss_func = losses.TripletMarginLoss(margin=margin_num, distance=distance, reducer=
#loss_func = losses.ContrastiveLoss()
#loss_func = losses.ArcFaceLoss(num_classes = 10, embedding_size = 128)
mining_func = miners.TripletMarginMiner(
    margin=margin_num, distance=distance, type_of_triplets="semihard"
)


accuracy_calculator = AccuracyCalculator(include=("precision_at_1",), k=1)
### pytorch-metric-learning stuff ###


loss_list_all = []
accuracies_list = []
accuracies = test(dataset1, dataset2, model, accuracy_calculator)
accuracies_list.append((0, accuracies))

for epoch in range(1, num_epochs + 1):
    loss_list = train(model, loss_func, mining_func, device, train_loader, optimize
    accuracies = test(dataset1, dataset2, model, accuracy_calculator)
    loss_list_all += loss_list
    accuracies_list.append((len(loss_list_all), accuracies))
```

```
100%|██████████| 1875/1875 [00:07<00:00, 242.70it/s]
100%|██████████| 313/313 [00:01<00:00, 231.07it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.850200
Epoch 1 Iteration 0: Loss = 0.09754770249128342, Number of mined triplets = 451984
Epoch 1 Iteration 20: Loss = 0.10957798361778259, Number of mined triplets = 154955
Epoch 1 Iteration 40: Loss = 0.09865615516901016, Number of mined triplets = 106186
Epoch 1 Iteration 60: Loss = 0.09553137421607971, Number of mined triplets = 82460
Epoch 1 Iteration 80: Loss = 0.09661511331796646, Number of mined triplets = 87020
Epoch 1 Iteration 100: Loss = 0.09657326340675354, Number of mined triplets = 68473
Epoch 1 Iteration 120: Loss = 0.09991620481014252, Number of mined triplets = 97945
Epoch 1 Iteration 140: Loss = 0.09412653744220734, Number of mined triplets = 62071
Epoch 1 Iteration 160: Loss = 0.0970236212015152, Number of mined triplets = 92341
Epoch 1 Iteration 180: Loss = 0.09568595886230469, Number of mined triplets = 61972
Epoch 1 Iteration 200: Loss = 0.0939253568649292, Number of mined triplets = 61076
Epoch 1 Iteration 220: Loss = 0.09466403722763062, Number of mined triplets = 74004
100%|██████████| 1875/1875 [00:04<00:00, 392.31it/s]
100%|██████████| 313/313 [00:01<00:00, 229.43it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.838100
Epoch 2 Iteration 0: Loss = 0.09372390061616898, Number of mined triplets = 57331
Epoch 2 Iteration 20: Loss = 0.09346999228000641, Number of mined triplets = 70577
Epoch 2 Iteration 40: Loss = 0.09327021986246109, Number of mined triplets = 64304
Epoch 2 Iteration 60: Loss = 0.09312213212251663, Number of mined triplets = 55825
Epoch 2 Iteration 80: Loss = 0.09189867973327637, Number of mined triplets = 71599
Epoch 2 Iteration 100: Loss = 0.09229299426078796, Number of mined triplets = 72400
Epoch 2 Iteration 120: Loss = 0.09087519347667694, Number of mined triplets = 40323
Epoch 2 Iteration 140: Loss = 0.09227097779512405, Number of mined triplets = 62667
Epoch 2 Iteration 160: Loss = 0.09483222663402557, Number of mined triplets = 77880
Epoch 2 Iteration 180: Loss = 0.0918414369225502, Number of mined triplets = 61942
Epoch 2 Iteration 200: Loss = 0.09294197708368301, Number of mined triplets = 55118
Epoch 2 Iteration 220: Loss = 0.09119900315999985, Number of mined triplets = 62507
```

```
100%|██████████| 1875/1875 [00:07<00:00, 251.68it/s]
100%|██████████| 313/313 [00:01<00:00, 240.47it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.851800
Epoch 3 Iteration 0: Loss = 0.09193644672632217, Number of mined triplets = 58731
Epoch 3 Iteration 20: Loss = 0.09411627054214478, Number of mined triplets = 76956
Epoch 3 Iteration 40: Loss = 0.09199028462171555, Number of mined triplets = 70461
Epoch 3 Iteration 60: Loss = 0.09348320960998535, Number of mined triplets = 65257
Epoch 3 Iteration 80: Loss = 0.09445706009864807, Number of mined triplets = 54764
Epoch 3 Iteration 100: Loss = 0.09169606119394302, Number of mined triplets = 44379
Epoch 3 Iteration 120: Loss = 0.09101670235395432, Number of mined triplets = 44727
Epoch 3 Iteration 140: Loss = 0.09159655123949051, Number of mined triplets = 54809
Epoch 3 Iteration 160: Loss = 0.0914444774389267, Number of mined triplets = 66220
Epoch 3 Iteration 180: Loss = 0.09265695512294769, Number of mined triplets = 75982
Epoch 3 Iteration 200: Loss = 0.09186442196369171, Number of mined triplets = 55656
Epoch 3 Iteration 220: Loss = 0.09197758883237839, Number of mined triplets = 55708
100%|██████████| 1875/1875 [00:04<00:00, 390.91it/s]
100%|██████████| 313/313 [00:00<00:00, 352.74it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.853700
Epoch 4 Iteration 0: Loss = 0.09212248772382736, Number of mined triplets = 64263
Epoch 4 Iteration 20: Loss = 0.09238816052675247, Number of mined triplets = 69271
Epoch 4 Iteration 40: Loss = 0.09147343784570694, Number of mined triplets = 52132
Epoch 4 Iteration 60: Loss = 0.09213607758283615, Number of mined triplets = 76924
Epoch 4 Iteration 80: Loss = 0.09294609725475311, Number of mined triplets = 66898
Epoch 4 Iteration 100: Loss = 0.09298276901245117, Number of mined triplets = 63511
Epoch 4 Iteration 120: Loss = 0.09190811961889267, Number of mined triplets = 67845
Epoch 4 Iteration 140: Loss = 0.09171900898218155, Number of mined triplets = 52127
Epoch 4 Iteration 160: Loss = 0.09244231134653091, Number of mined triplets = 83329
Epoch 4 Iteration 180: Loss = 0.09241315722465515, Number of mined triplets = 60316
Epoch 4 Iteration 200: Loss = 0.09062906354665756, Number of mined triplets = 51053
Epoch 4 Iteration 220: Loss = 0.09073680639266968, Number of mined triplets = 64410
100%|██████████| 1875/1875 [00:04<00:00, 397.68it/s]
100%|██████████| 313/313 [00:01<00:00, 229.99it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.856300
Epoch 5 Iteration 0: Loss = 0.09120571613311768, Number of mined triplets = 48525
Epoch 5 Iteration 20: Loss = 0.09207625687122345, Number of mined triplets = 54579
Epoch 5 Iteration 40: Loss = 0.0899977758526802, Number of mined triplets = 53302
Epoch 5 Iteration 60: Loss = 0.09261218458414078, Number of mined triplets = 57357
Epoch 5 Iteration 80: Loss = 0.09184911847114563, Number of mined triplets = 56173
Epoch 5 Iteration 100: Loss = 0.09227092564105988, Number of mined triplets = 57243
Epoch 5 Iteration 120: Loss = 0.09055743366479874, Number of mined triplets = 55664
Epoch 5 Iteration 140: Loss = 0.0901634618639946, Number of mined triplets = 52723
Epoch 5 Iteration 160: Loss = 0.09376528859138489, Number of mined triplets = 71070
Epoch 5 Iteration 180: Loss = 0.0914669781923294, Number of mined triplets = 55209
Epoch 5 Iteration 200: Loss = 0.09322971850633621, Number of mined triplets = 66335
Epoch 5 Iteration 220: Loss = 0.09226889163255692, Number of mined triplets = 71493
100%|██████████| 1875/1875 [00:04<00:00, 379.10it/s]
100%|██████████| 313/313 [00:01<00:00, 239.14it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.856600
Epoch 6 Iteration 0: Loss = 0.09295734763145447, Number of mined triplets = 63674
Epoch 6 Iteration 20: Loss = 0.09273751080036163, Number of mined triplets = 73828
Epoch 6 Iteration 40: Loss = 0.09061414748430252, Number of mined triplets = 59124
Epoch 6 Iteration 60: Loss = 0.0930802971124649, Number of mined triplets = 61543
Epoch 6 Iteration 80: Loss = 0.09064231812953949, Number of mined triplets = 45620
Epoch 6 Iteration 100: Loss = 0.09207115322351456, Number of mined triplets = 48252
Epoch 6 Iteration 120: Loss = 0.09431219100952148, Number of mined triplets = 74167
Epoch 6 Iteration 140: Loss = 0.09158093482255936, Number of mined triplets = 49354
Epoch 6 Iteration 160: Loss = 0.09198856353759766, Number of mined triplets = 61992
Epoch 6 Iteration 180: Loss = 0.09105170518159866, Number of mined triplets = 59241
Epoch 6 Iteration 200: Loss = 0.09235172718763351, Number of mined triplets = 53901
Epoch 6 Iteration 220: Loss = 0.09219400584697723, Number of mined triplets = 58154
100%|██████████| 1875/1875 [00:04<00:00, 381.54it/s]
100%|██████████| 313/313 [00:01<00:00, 233.26it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.853400
Epoch 7 Iteration 0: Loss = 0.0927167609333992, Number of mined triplets = 58184
Epoch 7 Iteration 20: Loss = 0.09209046512842178, Number of mined triplets = 64282
Epoch 7 Iteration 40: Loss = 0.09124661237001419, Number of mined triplets = 60667
Epoch 7 Iteration 60: Loss = 0.09171270579099655, Number of mined triplets = 59441
Epoch 7 Iteration 80: Loss = 0.09002319723367691, Number of mined triplets = 44070
Epoch 7 Iteration 100: Loss = 0.09191009402275085, Number of mined triplets = 62596
Epoch 7 Iteration 120: Loss = 0.09081359207630157, Number of mined triplets = 59310
Epoch 7 Iteration 140: Loss = 0.09121380746364594, Number of mined triplets = 59908
Epoch 7 Iteration 160: Loss = 0.0923006683588028, Number of mined triplets = 65126
Epoch 7 Iteration 180: Loss = 0.09109989553689957, Number of mined triplets = 53216
Epoch 7 Iteration 200: Loss = 0.09325575828552246, Number of mined triplets = 66468
Epoch 7 Iteration 220: Loss = 0.09089145809412003, Number of mined triplets = 56939
100%|██████████| 1875/1875 [00:04<00:00, 385.60it/s]
100%|██████████| 313/313 [00:01<00:00, 235.79it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.852700
Epoch 8 Iteration 0: Loss = 0.0907420665025711, Number of mined triplets = 56209
Epoch 8 Iteration 20: Loss = 0.09188880026340485, Number of mined triplets = 61171
Epoch 8 Iteration 40: Loss = 0.09203047305345535, Number of mined triplets = 49083
Epoch 8 Iteration 60: Loss = 0.09130171686410904, Number of mined triplets = 47598
Epoch 8 Iteration 80: Loss = 0.0940096378326416, Number of mined triplets = 67955
Epoch 8 Iteration 100: Loss = 0.09059802442789078, Number of mined triplets = 54200
Epoch 8 Iteration 120: Loss = 0.09189173579216003, Number of mined triplets = 60639
Epoch 8 Iteration 140: Loss = 0.09122709929943085, Number of mined triplets = 60540
Epoch 8 Iteration 160: Loss = 0.09251773357391357, Number of mined triplets = 48698
Epoch 8 Iteration 180: Loss = 0.09121987223625183, Number of mined triplets = 65820
Epoch 8 Iteration 200: Loss = 0.09180513024330139, Number of mined triplets = 54896
Epoch 8 Iteration 220: Loss = 0.0914236530661583, Number of mined triplets = 66848
100%|██████████| 1875/1875 [00:04<00:00, 383.83it/s]
100%|██████████| 313/313 [00:01<00:00, 246.66it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.851000
Epoch 9 Iteration 0: Loss = 0.09341340512037277, Number of mined triplets = 61568
Epoch 9 Iteration 20: Loss = 0.09130606800317764, Number of mined triplets = 63144
Epoch 9 Iteration 40: Loss = 0.09383189678192139, Number of mined triplets = 72100
Epoch 9 Iteration 60: Loss = 0.09123191237449646, Number of mined triplets = 51272
Epoch 9 Iteration 80: Loss = 0.09237077087163925, Number of mined triplets = 69975
Epoch 9 Iteration 100: Loss = 0.09234242141246796, Number of mined triplets = 62140
Epoch 9 Iteration 120: Loss = 0.09212098270654678, Number of mined triplets = 54449
Epoch 9 Iteration 140: Loss = 0.09131086617708206, Number of mined triplets = 54728
Epoch 9 Iteration 160: Loss = 0.09229633957147598, Number of mined triplets = 69846
Epoch 9 Iteration 180: Loss = 0.0911346971988678, Number of mined triplets = 50783
Epoch 9 Iteration 200: Loss = 0.09027215093374252, Number of mined triplets = 36703
Epoch 9 Iteration 220: Loss = 0.09296669811010361, Number of mined triplets = 57827
100%|██████████| 1875/1875 [00:04<00:00, 385.51it/s]
100%|██████████| 313/313 [00:01<00:00, 234.83it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.854000
Epoch 10 Iteration 0: Loss = 0.09218641370534897, Number of mined triplets = 64433
Epoch 10 Iteration 20: Loss = 0.09257309138774872, Number of mined triplets = 60049
Epoch 10 Iteration 40: Loss = 0.09115868806838989, Number of mined triplets = 57207
Epoch 10 Iteration 60: Loss = 0.09121999144554138, Number of mined triplets = 48220
Epoch 10 Iteration 80: Loss = 0.09117040038108826, Number of mined triplets = 47701
Epoch 10 Iteration 100: Loss = 0.08902604877948761, Number of mined triplets = 43113
Epoch 10 Iteration 120: Loss = 0.09001786261796951, Number of mined triplets = 46889
Epoch 10 Iteration 140: Loss = 0.09149924665689468, Number of mined triplets = 63451
Epoch 10 Iteration 160: Loss = 0.09170305728912354, Number of mined triplets = 65630
Epoch 10 Iteration 180: Loss = 0.09154872596263885, Number of mined triplets = 61008
Epoch 10 Iteration 200: Loss = 0.09248603135347366, Number of mined triplets = 71565
Epoch 10 Iteration 220: Loss = 0.09328345954418182, Number of mined triplets = 48378
100%|██████████| 1875/1875 [00:07<00:00, 247.56it/s]
100%|██████████| 313/313 [00:00<00:00, 340.49it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.858900
Epoch 11 Iteration 0: Loss = 0.08954893052577972, Number of mined triplets = 43841
Epoch 11 Iteration 20: Loss = 0.0903269499540329, Number of mined triplets = 51597
Epoch 11 Iteration 40: Loss = 0.09065651148557663, Number of mined triplets = 56423
Epoch 11 Iteration 60: Loss = 0.09089360386133194, Number of mined triplets = 50074
Epoch 11 Iteration 80: Loss = 0.08964370936155319, Number of mined triplets = 43459
Epoch 11 Iteration 100: Loss = 0.09266118705272675, Number of mined triplets = 68401
Epoch 11 Iteration 120: Loss = 0.09205140918493271, Number of mined triplets = 55823
Epoch 11 Iteration 140: Loss = 0.0925883799791336, Number of mined triplets = 62840
Epoch 11 Iteration 160: Loss = 0.09147879481315613, Number of mined triplets = 64320
Epoch 11 Iteration 180: Loss = 0.0930623933672905, Number of mined triplets = 65005
Epoch 11 Iteration 200: Loss = 0.09076937288045883, Number of mined triplets = 49451
Epoch 11 Iteration 220: Loss = 0.08982081711292267, Number of mined triplets = 53569
100%|██████████| 1875/1875 [00:04<00:00, 392.99it/s]
100%|██████████| 313/313 [00:01<00:00, 241.20it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.858400
Epoch 12 Iteration 0: Loss = 0.08986499905586243, Number of mined triplets = 42139
Epoch 12 Iteration 20: Loss = 0.09280005842447281, Number of mined triplets = 51521
Epoch 12 Iteration 40: Loss = 0.09229465574026108, Number of mined triplets = 58347
Epoch 12 Iteration 60: Loss = 0.09266294538974762, Number of mined triplets = 52308
Epoch 12 Iteration 80: Loss = 0.09201957285404205, Number of mined triplets = 46539
Epoch 12 Iteration 100: Loss = 0.0931968241930008, Number of mined triplets = 70727
Epoch 12 Iteration 120: Loss = 0.08984184265136719, Number of mined triplets = 51033
Epoch 12 Iteration 140: Loss = 0.09129825234413147, Number of mined triplets = 43088
Epoch 12 Iteration 160: Loss = 0.0923999473452568, Number of mined triplets = 58347
Epoch 12 Iteration 180: Loss = 0.0888424813747406, Number of mined triplets = 54768
Epoch 12 Iteration 200: Loss = 0.09080719947814941, Number of mined triplets = 46391
Epoch 12 Iteration 220: Loss = 0.09221640229225159, Number of mined triplets = 63106

100%|██████████| 1875/1875 [00:04<00:00, 397.04it/s]
100%|██████████| 313/313 [00:00<00:00, 335.43it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.857000
Epoch 13 Iteration 0: Loss = 0.09204115718603134, Number of mined triplets = 49041
Epoch 13 Iteration 20: Loss = 0.09181886911392212, Number of mined triplets = 57516
Epoch 13 Iteration 40: Loss = 0.08967522531747818, Number of mined triplets = 37822
Epoch 13 Iteration 60: Loss = 0.09250519424676895, Number of mined triplets = 57281
Epoch 13 Iteration 80: Loss = 0.090220607817173, Number of mined triplets = 46674
Epoch 13 Iteration 100: Loss = 0.09187328070402145, Number of mined triplets = 51947
Epoch 13 Iteration 120: Loss = 0.08951354771852493, Number of mined triplets = 48810
Epoch 13 Iteration 140: Loss = 0.09232193976640701, Number of mined triplets = 73729
Epoch 13 Iteration 160: Loss = 0.09144409000873566, Number of mined triplets = 60411
Epoch 13 Iteration 180: Loss = 0.09194373339414597, Number of mined triplets = 63372
Epoch 13 Iteration 200: Loss = 0.08986032754182816, Number of mined triplets = 50308
Epoch 13 Iteration 220: Loss = 0.09061494469642639, Number of mined triplets = 47408

100%|██████████| 1875/1875 [00:04<00:00, 389.80it/s]
100%|██████████| 313/313 [00:00<00:00, 338.45it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.857600
Epoch 14 Iteration 0: Loss = 0.09076526761054993, Number of mined triplets = 58815
Epoch 14 Iteration 20: Loss = 0.09178665280342102, Number of mined triplets = 52242
Epoch 14 Iteration 40: Loss = 0.09040797501802444, Number of mined triplets = 61390
Epoch 14 Iteration 60: Loss = 0.09143292903900146, Number of mined triplets = 49210
Epoch 14 Iteration 80: Loss = 0.0923723578453064, Number of mined triplets = 58122
Epoch 14 Iteration 100: Loss = 0.0927417129278183, Number of mined triplets = 65017
Epoch 14 Iteration 120: Loss = 0.09090641885995865, Number of mined triplets = 49585
Epoch 14 Iteration 140: Loss = 0.09091834723949432, Number of mined triplets = 50601
Epoch 14 Iteration 160: Loss = 0.0897802785038948, Number of mined triplets = 44387
Epoch 14 Iteration 180: Loss = 0.09034387022256851, Number of mined triplets = 51144
Epoch 14 Iteration 200: Loss = 0.093791164457798, Number of mined triplets = 59333
Epoch 14 Iteration 220: Loss = 0.09304246306419373, Number of mined triplets = 59862

100%|██████████| 1875/1875 [00:07<00:00, 249.30it/s]
100%|██████████| 313/313 [00:01<00:00, 241.78it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.862100
Epoch 15 Iteration 0: Loss = 0.09128481894731522, Number of mined triplets = 58772
Epoch 15 Iteration 20: Loss = 0.09251119941473007, Number of mined triplets = 55542
Epoch 15 Iteration 40: Loss = 0.09148107469081879, Number of mined triplets = 63577
Epoch 15 Iteration 60: Loss = 0.09032467752695084, Number of mined triplets = 45636
Epoch 15 Iteration 80: Loss = 0.0884544849395752, Number of mined triplets = 47091
Epoch 15 Iteration 100: Loss = 0.09192704409360886, Number of mined triplets = 46080
Epoch 15 Iteration 120: Loss = 0.0919080525636673, Number of mined triplets = 51366
Epoch 15 Iteration 140: Loss = 0.0928633064031601, Number of mined triplets = 55287
Epoch 15 Iteration 160: Loss = 0.09171327203512192, Number of mined triplets = 45497
Epoch 15 Iteration 180: Loss = 0.09064455330371857, Number of mined triplets = 57435
Epoch 15 Iteration 200: Loss = 0.09137553721666336, Number of mined triplets = 47053
Epoch 15 Iteration 220: Loss = 0.09057556092739105, Number of mined triplets = 57386
100%|██████████| 1875/1875 [00:07<00:00, 253.64it/s]
100%|██████████| 313/313 [00:01<00:00, 228.49it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.862800
Epoch 16 Iteration 0: Loss = 0.09107762575149536, Number of mined triplets = 48866
Epoch 16 Iteration 20: Loss = 0.09170708805322647, Number of mined triplets = 42369
Epoch 16 Iteration 40: Loss = 0.09100520610809326, Number of mined triplets = 56728
Epoch 16 Iteration 60: Loss = 0.09328741580247879, Number of mined triplets = 60833
Epoch 16 Iteration 80: Loss = 0.09103350341320038, Number of mined triplets = 56370
Epoch 16 Iteration 100: Loss = 0.09158801287412643, Number of mined triplets = 36206
Epoch 16 Iteration 120: Loss = 0.09005969762802124, Number of mined triplets = 47152
Epoch 16 Iteration 140: Loss = 0.09093885868787766, Number of mined triplets = 52671
Epoch 16 Iteration 160: Loss = 0.09170272946357727, Number of mined triplets = 55021
Epoch 16 Iteration 180: Loss = 0.09081428498029709, Number of mined triplets = 52887
Epoch 16 Iteration 200: Loss = 0.09184485673904419, Number of mined triplets = 56156
Epoch 16 Iteration 220: Loss = 0.09002818167209625, Number of mined triplets = 49683
100%|██████████| 1875/1875 [00:04<00:00, 406.37it/s]
100%|██████████| 313/313 [00:00<00:00, 321.15it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.856200
Epoch 17 Iteration 0: Loss = 0.09271501749753952, Number of mined triplets = 51035
Epoch 17 Iteration 20: Loss = 0.0906735360622406, Number of mined triplets = 49708
Epoch 17 Iteration 40: Loss = 0.09209480881690979, Number of mined triplets = 55542
Epoch 17 Iteration 60: Loss = 0.09217363595962524, Number of mined triplets = 54107
Epoch 17 Iteration 80: Loss = 0.09100080281496048, Number of mined triplets = 52441
Epoch 17 Iteration 100: Loss = 0.09123574197292328, Number of mined triplets = 51635
Epoch 17 Iteration 120: Loss = 0.09256389737129211, Number of mined triplets = 48513
Epoch 17 Iteration 140: Loss = 0.09276492893695831, Number of mined triplets = 61463
Epoch 17 Iteration 160: Loss = 0.09196581691503525, Number of mined triplets = 50168
Epoch 17 Iteration 180: Loss = 0.09209530055522919, Number of mined triplets = 76208
Epoch 17 Iteration 200: Loss = 0.09087599813938141, Number of mined triplets = 41318
Epoch 17 Iteration 220: Loss = 0.09190346300601959, Number of mined triplets = 52791
100%|██████████| 1875/1875 [00:04<00:00, 378.63it/s]
100%|██████████| 313/313 [00:01<00:00, 223.91it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.859400
Epoch 18 Iteration 0: Loss = 0.09054086357355118, Number of mined triplets = 52690
Epoch 18 Iteration 20: Loss = 0.09024777263402939, Number of mined triplets = 57984
Epoch 18 Iteration 40: Loss = 0.09074240922927856, Number of mined triplets = 53374
Epoch 18 Iteration 60: Loss = 0.09126794338226318, Number of mined triplets = 58218
Epoch 18 Iteration 80: Loss = 0.0897250771522522, Number of mined triplets = 47457
Epoch 18 Iteration 100: Loss = 0.090654157102108, Number of mined triplets = 56682
Epoch 18 Iteration 120: Loss = 0.09170309454202652, Number of mined triplets = 58435
Epoch 18 Iteration 140: Loss = 0.0920722559094429, Number of mined triplets = 57693
Epoch 18 Iteration 160: Loss = 0.0902276337146759, Number of mined triplets = 49520
Epoch 18 Iteration 180: Loss = 0.09338454157114029, Number of mined triplets = 83258
Epoch 18 Iteration 200: Loss = 0.09258169680833817, Number of mined triplets = 65867
Epoch 18 Iteration 220: Loss = 0.08900555223226547, Number of mined triplets = 43794
100%|██████████| 1875/1875 [00:07<00:00, 249.30it/s]
100%|██████████| 313/313 [00:01<00:00, 230.28it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.862900
Epoch 19 Iteration 0: Loss = 0.09094059467315674, Number of mined triplets = 54445
Epoch 19 Iteration 20: Loss = 0.08971578627824783, Number of mined triplets = 41556
Epoch 19 Iteration 40: Loss = 0.09169431030750275, Number of mined triplets = 49666
Epoch 19 Iteration 60: Loss = 0.09168478846549988, Number of mined triplets = 54721
Epoch 19 Iteration 80: Loss = 0.09210909903049469, Number of mined triplets = 49277
Epoch 19 Iteration 100: Loss = 0.09134359657764435, Number of mined triplets = 50649
Epoch 19 Iteration 120: Loss = 0.09089117497205734, Number of mined triplets = 54730
Epoch 19 Iteration 140: Loss = 0.09095916897058487, Number of mined triplets = 50600
Epoch 19 Iteration 160: Loss = 0.08963069319725037, Number of mined triplets = 46058
Epoch 19 Iteration 180: Loss = 0.09170113503932953, Number of mined triplets = 58836
Epoch 19 Iteration 200: Loss = 0.09046555310487747, Number of mined triplets = 47722
Epoch 19 Iteration 220: Loss = 0.09192623943090439, Number of mined triplets = 57359
100%|██████████| 1875/1875 [00:04<00:00, 382.91it/s]
100%|██████████| 313/313 [00:01<00:00, 205.83it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.862400
Epoch 20 Iteration 0: Loss = 0.09269391000270844, Number of mined triplets = 61550
Epoch 20 Iteration 20: Loss = 0.09150198847055435, Number of mined triplets = 53650
Epoch 20 Iteration 40: Loss = 0.09385237842798233, Number of mined triplets = 66497
Epoch 20 Iteration 60: Loss = 0.09018287807703018, Number of mined triplets = 48674
Epoch 20 Iteration 80: Loss = 0.09035464376211166, Number of mined triplets = 44588
Epoch 20 Iteration 100: Loss = 0.0937512144446373, Number of mined triplets = 48209
Epoch 20 Iteration 120: Loss = 0.09116949886083603, Number of mined triplets = 45239
Epoch 20 Iteration 140: Loss = 0.0912722572684288, Number of mined triplets = 53677
Epoch 20 Iteration 160: Loss = 0.09092972427606583, Number of mined triplets = 54030
Epoch 20 Iteration 180: Loss = 0.09223844110965729, Number of mined triplets = 52430
Epoch 20 Iteration 200: Loss = 0.0920858383178711, Number of mined triplets = 53654
Epoch 20 Iteration 220: Loss = 0.09086690843105316, Number of mined triplets = 45641
100%|██████████| 1875/1875 [00:07<00:00, 245.43it/s]
100%|██████████| 313/313 [00:00<00:00, 339.42it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.856400
```

In [7]:

```python
import matplotlib.pyplot as plt

loss_iters = list(range(len(loss_list_all)))
loss_vals = loss_list_all
```

```
acc_iters = [x[0] for x in accuracies_list]
acc_vals = [x[1]['precision_at_1'] for x in accuracies_list]

fig, ax1 = plt.subplots()

# Left: training loss
ax1.set_xlabel('training iter')
ax1.set_ylabel('training loss', color='blue')
ax1.plot(loss_iters, loss_vals, color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# Right: testing accuracy
ax2 = ax1.twinx()
ax2.set_ylabel('testing accuracy', color='green')
ax2.plot(acc_iters, acc_vals, 'o-', color='green')
ax2.tick_params(axis='y', labelcolor='green')

plt.title('Training Loss and Testing Accuracy')
plt.show()
```

(f) Change the margin of `losses.TripletMarginLoss` from 0.2 to 10, and re-train the ORIGINAL model (without the modification in (e)) for 2 epochs.

1. Report the final test accuracy; (2 points)

2. Compare with (a): what and why is the difference? (5 points)

In [9]:

```
# your code here
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.fc1 = nn.Linear(9216, 128)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
        x = torch.flatten(x, 1)
        # print(x.shape) # should be [batch_size, 9216]
        x = self.fc1(x)
        return x


### Do the training ###
def train(model, loss_func, mining_func, device, train_loader, optimizer, epoch):
    model.train()
    loss_list = []
    for batch_idx, (data, labels) in enumerate(train_loader):
        data, labels = data.to(device), labels.to(device)
        optimizer.zero_grad()
        embeddings = model(data)
        indices_tuple = mining_func(embeddings, labels)
        loss = loss_func(embeddings, labels, indices_tuple)
        loss.backward()
        optimizer.step()
        if batch_idx % 20 == 0:
            print(
                "Epoch {} Iteration {}: Loss = {}, Number of mined triplets = {}".f
                    epoch, batch_idx, loss, mining_func.num_triplets
                )
            )
        loss_list.append(loss.item())
    return loss_list


### convenient function from pytorch-metric-learning ###
def get_all_embeddings(dataset, model):
    tester = testers.BaseTester()
    return tester.get_all_embeddings(dataset, model)


### compute accuracy using AccuracyCalculator from pytorch-metric-learning ###
def test(train_set, test_set, model, accuracy_calculator):
    train_embeddings, train_labels = get_all_embeddings(train_set, model)
    test_embeddings, test_labels = get_all_embeddings(test_set, model)
    train_labels = train_labels.squeeze(1)
```

```
    test_labels = test_labels.squeeze(1)
    print("Computing accuracy")
    accuracies = accuracy_calculator.get_accuracy(
        test_embeddings, test_labels, train_embeddings, train_labels, False
    )
    print("Test set accuracy (Precision@1) = {:f}".format(accuracies["precision_at_
    return accuracies


device = torch.device("cuda")

transform = transforms.Compose(
    [transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))]
)

batch_size = 256

dataset1 = datasets.FashionMNIST(".", train=True, download=True, transform=transfor
dataset2 = datasets.FashionMNIST(".", train=False, transform=transform)
train_loader = torch.utils.data.DataLoader(dataset1, batch_size=256, shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset2, batch_size=256)

model = Net().to(device)

optimizer = optim.Adam(model.parameters(), lr=0.01)
num_epochs = 2

### pytorch-metric-learning stuff ###
distance = distances.CosineSimilarity()
#distance = distances.LpDistance()
reducer = reducers.ThresholdReducer(low=0)

margin_num = 10
loss_func = losses.TripletMarginLoss(margin=margin_num, distance=distance, reducer=
#loss_func = losses.ContrastiveLoss()
#loss_func = losses.ArcFaceLoss(num_classes = 10, embedding_size = 128)
mining_func = miners.TripletMarginMiner(
    margin=margin_num, distance=distance, type_of_triplets="semihard"
)


accuracy_calculator = AccuracyCalculator(include=("precision_at_1",), k=1)
### pytorch-metric-learning stuff ###


loss_list_all = []
accuracies_list = []
accuracies = test(dataset1, dataset2, model, accuracy_calculator)
accuracies_list.append((0, accuracies))

for epoch in range(1, num_epochs + 1):
    loss_list = train(model, loss_func, mining_func, device, train_loader, optimize
    accuracies = test(dataset1, dataset2, model, accuracy_calculator)
    loss_list_all += loss_list
    accuracies_list.append((len(loss_list_all), accuracies))
```

```
100%|██████████| 1875/1875 [00:05<00:00, 369.12it/s]
100%|██████████| 313/313 [00:01<00:00, 300.42it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.838700
Epoch 1 Iteration 0: Loss = 9.845601081848145, Number of mined triplets = 1104340
Epoch 1 Iteration 20: Loss = 8.931994438171387, Number of mined triplets = 1302604
Epoch 1 Iteration 40: Loss = 8.820704460144043, Number of mined triplets = 1284401
Epoch 1 Iteration 60: Loss = 8.839637756347656, Number of mined triplets = 1292406
Epoch 1 Iteration 80: Loss = 8.77319622039795, Number of mined triplets = 1275660
Epoch 1 Iteration 100: Loss = 8.785900115966797, Number of mined triplets = 1252238
Epoch 1 Iteration 120: Loss = 8.745498657226562, Number of mined triplets = 1272427
Epoch 1 Iteration 140: Loss = 8.784160614013672, Number of mined triplets = 1267037
Epoch 1 Iteration 160: Loss = 8.750509262084961, Number of mined triplets = 1262271
Epoch 1 Iteration 180: Loss = 8.822750091552734, Number of mined triplets = 1226876
Epoch 1 Iteration 200: Loss = 8.812994956970215, Number of mined triplets = 1256342
Epoch 1 Iteration 220: Loss = 8.831721305847168, Number of mined triplets = 1317946
100%|██████████| 1875/1875 [00:04<00:00, 378.81it/s]
100%|██████████| 313/313 [00:00<00:00, 318.45it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.787500
Epoch 2 Iteration 0: Loss = 8.766812324523926, Number of mined triplets = 1248093
Epoch 2 Iteration 20: Loss = 8.786815643310547, Number of mined triplets = 1260610
Epoch 2 Iteration 40: Loss = 8.794672012329102, Number of mined triplets = 1279386
Epoch 2 Iteration 60: Loss = 8.78492259979248, Number of mined triplets = 1267682
Epoch 2 Iteration 80: Loss = 8.76754379272461, Number of mined triplets = 1247209
Epoch 2 Iteration 100: Loss = 8.89224624633789, Number of mined triplets = 1264796
Epoch 2 Iteration 120: Loss = 8.726724624633789, Number of mined triplets = 1326344
Epoch 2 Iteration 140: Loss = 8.797096252441406, Number of mined triplets = 1232170
Epoch 2 Iteration 160: Loss = 8.788171768188477, Number of mined triplets = 1230285
Epoch 2 Iteration 180: Loss = 8.837019920349121, Number of mined triplets = 1267024
Epoch 2 Iteration 200: Loss = 8.741302490234375, Number of mined triplets = 1273278
Epoch 2 Iteration 220: Loss = 8.799406051635742, Number of mined triplets = 1252789
100%|██████████| 1875/1875 [00:05<00:00, 359.56it/s]
100%|██████████| 313/313 [00:01<00:00, 284.11it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.790700
```

1. The final accuracy is %79
2. This is different from the first time by a 10 percent margin of accuracy. The margin is larger which would explain the decrease in accuracy.

(g) Change the miner property to `type_of_triplets="hard"` (which is for hard nagetive mining instead of looking for semi-hard negative samples) and re-train the ORIGINAL model (without the modification in (e) and (f)) for 2 epochs.

1. Report the final test accuracy; (2 points)
2. Plot the curves again; (1 points)
3. Compare with (a): what and why is the difference? (5 points)

In [16]:
```
model = Net().to(device)

optimizer = optim.Adam(model.parameters(), lr=0.01)
num_epochs = 2

### pytorch-metric-learning stuff ###
distance = distances.CosineSimilarity()
#distance = distances.LpDistance()
reducer = reducers.ThresholdReducer(low=0)
margin_num = 0.2
loss_func = losses.TripletMarginLoss(margin=margin_num, distance=distance, reducer=
mining_func = miners.TripletMarginMiner(
    margin=margin_num, distance=distance, type_of_triplets="hard"
)


accuracy_calculator = AccuracyCalculator(include=("precision_at_1",), k=1)
### pytorch-metric-learning stuff ###


loss_list_all = []
accuracies_list = []
accuracies = test(dataset1, dataset2, model, accuracy_calculator)
accuracies_list.append((0, accuracies))

for epoch in range(1, num_epochs + 1):
    loss_list = train(model, loss_func, mining_func, device, train_loader, optimize
    accuracies = test(dataset1, dataset2, model, accuracy_calculator)
    loss_list_all += loss_list
    accuracies_list.append((len(loss_list_all), accuracies))
```

```
100%|██████████| 1875/1875 [00:07<00:00, 244.29it/s]
100%|██████████| 313/313 [00:00<00:00, 330.81it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.848500
Epoch 1 Iteration 0: Loss = 0.2843831181526184, Number of mined triplets = 392806
Epoch 1 Iteration 20: Loss = 0.20095106959342957, Number of mined triplets = 270061
Epoch 1 Iteration 40: Loss = 0.20056191086769104, Number of mined triplets = 224313
Epoch 1 Iteration 60: Loss = 0.20040558278560638, Number of mined triplets = 218201
Epoch 1 Iteration 80: Loss = 0.20026598870754242, Number of mined triplets = 209439
Epoch 1 Iteration 100: Loss = 0.2001650333404541, Number of mined triplets = 179357
Epoch 1 Iteration 120: Loss = 0.20014558732509613, Number of mined triplets = 176516
Epoch 1 Iteration 140: Loss = 0.20012766122817993, Number of mined triplets = 172523
Epoch 1 Iteration 160: Loss = 0.20008529722690582, Number of mined triplets = 180616
Epoch 1 Iteration 180: Loss = 0.2000730186700821, Number of mined triplets = 161117
Epoch 1 Iteration 200: Loss = 0.20007188618183136, Number of mined triplets = 145842
Epoch 1 Iteration 220: Loss = 0.2000611573457718, Number of mined triplets = 153515
100%|██████████| 1875/1875 [00:06<00:00, 306.60it/s]
100%|██████████| 313/313 [00:01<00:00, 232.74it/s]
```

```
Computing accuracy
Test set accuracy (Precision@1) = 0.830000
Epoch 2 Iteration 0: Loss = 0.20004281401634216, Number of mined triplets = 115320
Epoch 2 Iteration 20: Loss = 0.20003724098205566, Number of mined triplets = 141901
Epoch 2 Iteration 40: Loss = 0.20003104209899902, Number of mined triplets = 161875
Epoch 2 Iteration 60: Loss = 0.20002947747707367, Number of mined triplets = 138163
Epoch 2 Iteration 80: Loss = 0.20002004504203796, Number of mined triplets = 162091
Epoch 2 Iteration 100: Loss = 0.20002613961696625, Number of mined triplets = 130090
Epoch 2 Iteration 120: Loss = 0.20001430809497833, Number of mined triplets = 118690
Epoch 2 Iteration 140: Loss = 0.20001164078712463, Number of mined triplets = 152385
Epoch 2 Iteration 160: Loss = 0.20001688599586487, Number of mined triplets = 142149
Epoch 2 Iteration 180: Loss = 0.20000898838043213, Number of mined triplets = 163222
Epoch 2 Iteration 200: Loss = 0.20001086592674255, Number of mined triplets = 198174
Epoch 2 Iteration 220: Loss = 0.2000076323747635, Number of mined triplets = 136408
100%|██████████| 1875/1875 [00:07<00:00, 243.00it/s]
100%|██████████| 313/313 [00:01<00:00, 239.51it/s]
Computing accuracy
Test set accuracy (Precision@1) = 0.827900
```

In [17]:

```python
loss_iters = list(range(len(loss_list_all)))
loss_vals = loss_list_all

acc_iters = [x[0] for x in accuracies_list]
acc_vals = [x[1]['precision_at_1'] for x in accuracies_list]

fig, ax1 = plt.subplots()

# Left: training loss
ax1.set_xlabel('training iter')
ax1.set_ylabel('training loss', color='blue')
ax1.plot(loss_iters, loss_vals, color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# Right: testing accuracy
ax2 = ax1.twinx()
ax2.set_ylabel('testing accuracy', color='green')
ax2.plot(acc_iters, acc_vals, 'o-', color='green')
ax2.tick_params(axis='y', labelcolor='green')

plt.title('Training Loss and Testing Accuracy')
plt.show()
```

1. Final accuracy of %82.79
2. `include the image here, by downloading  from the notebook and uploading it into the ./images folder, modify the path and run the block to reveal the image:`
3. The difference we can see here is the acute drop and lack of variation from setting the mode to hard

Reference

[1] DeTone, Daniel, Tomasz Malisiewicz, and Andrew Rabinovich. "Superpoint: Self-supervised interest point detection and description." *Proceedings of the IEEE conference on computer vision and pattern recognition workshops* . 2018.

[2] Choy, Christopher B., et al. "Universal correspondence network." *Advances in neural information processing systems* 29 (2016).